

RADIOGRAFÍA DEL MVP PRESIDENTE-ONLY — GovernArg

Qué es esto: descripción exacta de CÓMO FUNCIONA HOY la dinámica del juego, a partir del código. Sin propuestas ni balance.

Base: branch `redisenovisual` · commit `e8ffca3` · 232/232 tests verdes.

Método: 4 auditorías de solo lectura en paralelo (variables/resultados, acciones/grupos, ideología/asesores, sistemas muertos/contradicciones). Toda afirmación tiene referencia `archivo:función:línea` en las secciones 1-4.

Convención: FUNCIONA = tiene efecto mecánico real · DECORATIVO = existe en data/UI pero ningún engine lo consume · MUERTO = código inalcanzable.

Síntesis ejecutiva — respuestas directas a las 7 preguntas

1. Variables de estado (detalle en Sección 1)

El `GameState` (~40 campos, `types/game.ts:453`) tiene un núcleo pequeño que realmente gobierna la partida y una periferia grande de variables con efectos parciales o nulos:

Variable	Rol real hoy
popularidad	La variable del juego. Decay fijo ~10/turno, piso dual $\max(\text{pop}, 0.4 \times \text{apoyoGrupos} + 0.6 \times \text{pop})$ (utils/popularidad.ts:11-12) que hace que el apoyo grupal solo pueda subirla, nunca bajarla. Umbral de derrota <30 (×2 turnos). Peso mayor en la fórmula de votos.
presupuesto	Deflacionario por diseño: 3500 inicial, neto +150/turno (500 ingreso – 350 mantenimiento). Derrota <0 (×2). Peso moderado en votos (crecimiento vs historicalBudget[0]).
apoyo de grupos (18 subgrupos)	Se mueve solo por 8 acciones con explicitGroupEffects + interacciones (+2/+4/+15) + demandas (+10/+5). Feedback sobre popularidad vía el piso dual.
estabilidad	Sube/baja por eventos, estrategia de coalición y extremos ideológicos. No tiene derrota propia: solo co-condición del impeachment (que queda sombreado por la derrota de popularidad). Warning a <25 sin consecuencia directa.
legitimidad	Idem: modificada por eventos/elecciones, sin derrota propia ni consumidor fuerte.
conflicto social	Casi decorativo: chips "—" fijos en UI (sin series históricas), sin efectos mecánicos claros sobre otros sistemas.
apoyo legislativo	Null durante todo el año 1 de cada mandato; se asigna en legislativas (año 2T4) y queda fijo hasta la próxima elección. Su único peso real: bloquea reforma_impositiva (LS≥45, inalcanzable en año 1) y la penalidad legislativa de reformas.
intención de voto	No es variable persistente: se calcula solo en elección (electionSystem.ts), mezclando popularidad reciente (slice –4), apoyos grupales, estabilidad, presupuesto y actividad.
inflación / moneyPrintingCount	Solo vía emitir_dinero: warning a 3, salto fuerte a 5 (~20 pop/~300), derrota a 7 (inalcanzable en un mandato: máx ~4-5 emisiones).

Reset al reelegir (asimétrico y relevante): resetea objetivos, popularidad ($\text{pop} \times 0.7 + 30$ — **puede subirla**: 40→58), presupuesto (3500+10%), historicalBudget, pendingEffects, cooldowns de eventos y moneyPrintingCount. **No resetea**: estabilidad, legitimidad, ejes ideológicos, groupRelations — y **borra a los asesores sin aviso en el flujo** (`advisors=[]`).

2. Resultado del juego (detalle en Sección 1, Parte 2)

- **Derrotas realmente alcanzables (4 de 6):** `low\_popularity` (<30 ×2), `negative\_budget` (<0 ×2), `hyperinflation` (7 emisiones), `election\_loss` (<45% votos). `impeachment` está sombreado (mismos 2 turnos que popularidad, evaluado después → nunca ocurre) e `institutional\_coup` es imposible (clamp de apoyo ≥25).

- **Reelección:** fórmula de votos en `electionSystem.ts:17+` — componentes: popularidad reciente, apoyos grupales ponderados por influencia, estabilidad, crecimiento de presupuesto vs `historicalBudget[0]`, actividad del mandato, bono/malus de eventos. Umbral 45 (`MIN\_VOTES\_TO\_WIN`), duplicado hardcoded en la UI (`ReelectionChoiceModal.tsx:46,93`). El pasivo puro llega a ~49: la elección casi no filtra.

- **Victoria final:** objetivos del presidente con lógica AND (`victoryConditions.ts`), generados por mandato (el reset los borra), re-evaluados dentro de `finalizePresidentialCareer` (fix del off-by-one). En la práctica ~0%: `national-prosperity` (10000M + pop≥80) supera el techo teórico (~9900M) y `total-stability` exige 5 grupos a 75-85 contra la matriz de antagonismo.

3. Acciones políticas (detalle en Sección 2)

61 acciones en `actionRegistry.ts`. Cadena real de ejecución: `canExecuteAction` (restricciones: minBudget, cooldown, prerequisites; minLegislativeSupport/minGroupSupport sin datos efectivos) → efectos **directos** (budget, popularity, estabilidad, legitimidad, LS, grupos) → **indirectos** (mantenimiento diferido 15% en acciones ≥200M a 2-4 turnos; +10% ingreso por infraestructura ×3; penalidad legislativa) → **diferidos** (`pendingEffects`/`futureEffects` con delays; los de delay 5-6 se pierden al reelegir).

El apoyo grupal solo se mueve por **8 acciones con `explicitGroupEffects`**; el matcher textual está 100% muerto (sus 4 coincidencias reales viven en acciones que ya tienen efectos explícitos con prioridad). `affectedGroups` (53/61 acciones) es **tooltip-only**: el jugador ve "Apoyan/Se oponen" que nunca ocurre.

4. Grupos de interés — ¿es demasiado fácil tener a todos contentos? Sí (detalle y cuenta en Sección 2)

- 18 subgrupos con apoyo e influencia iniciales (`interestGroups.ts`); 14 de 18 jamás eran afectados por acciones antes del fix, hoy 10 reciben efectos vía las 8 acciones explícitas.
- Antagonismo real: la matriz completa está en la Sección 2, pero el **antagonismo efectivo solo castiga sobre las ganancias de 8 acciones**.
- Las interacciones son la válvula que rompe el balance: reunión **+2/turno/grupo a \$10**, sin cooldown compartido ni castigo. Castigo típico máximo por antagonismo: -6/turno a un grupo $\Rightarrow$ se compensa con 3 reuniones (\$30). No hay oportunidad costada de mantener a todos contentos: **cuesta \$30/turno comprar la paz total**, contra un ingreso de 500.
- Efecto perverso adicional: `subsidiros\_industriales` sube +8 a empresarios y +8 a sindicatos (antagonistas mutuos) en la misma acción — el objetivo "Estabilidad Total" (que pide empresarios 85 + sindicatos 85 + populares 80 + clase-media 80 + clase-alta 75) es alcanzable precisamente por eso.
- Doble vía de demandas asimétrica: cumplir ejecutando la acción da +10 **sin** castigo de antagonistas; el botón "satisfacer" da +5 **con** castigo. Incentiva siempre la vía acción.

5. Sistema ideológico — tiene dientes, pero es ciego (detalle en Sección 3)

- Ejes definidos en `axisEngine.ts` (apertura/cierre, intervención/mercado, etc. — ver Sección 3); las acciones llevan tags por eje y el perfil del jugador se calcula acumulando ejecuciones.
- **Consecuencias mecánicas reales:** `effectivenessMultiplier` $\times 1.10$ -1.20 por afinidad acción-perfil (`actionEffects.ts:72`), `actionCostModifier` de $\pm 1\%$ (UI sugiere extremos graves; en costo es simbólico), y en extremos ($|e| \geq 80$): estabilidad $\blacksquare 5$ y relaciones grupales ± 10 con warnings alarmistas que describen castigo puro donde hay trade-off.
- **Ciego durante la partida:** el perfil solo se ve en `LegacyScreen` (pantalla final); `IndicatorsPanel` no muestra ejes pese a que el plan de implementación lo preveía.
- **Deriva monótona:** las pasivas de arquetipo empujan el eje **cada turno** sin compensación (`archetypeEngine.ts:67-70`): el arquetipo solo lleva un eje al extremo en ~40 turnos aunque el jugador no haga nada.

6. Asesores — dos bonos reales, el resto es escaparate (detalle en Sección 3)

- Listado completo y reglas de contratación (máx 2, anti-duplicados, \$300-500M, recalcula acciones al instante) en Sección 3.
- **FUNCIONA:** `bonusActions` (+1 acción) y el $\times 1.2$ de especialidad que multiplica solo `popularityChange` (no budget ni grupos, aunque la tarjeta insinúa "efectividad" general).
- **DISPLAY-ONLY:** `groupBonuses`, `policyModifiers`, `popularityEffect` — la UI los grafica (" +30% efectividad en economía", " +15% relación con empresarios") y ningún engine los lee. El jugador paga millones por estadísticas decorativas.
- **Bug de mapeo:** dos especialidades de la data ("Seguridad Pública", "Políticas Sociales") no coinciden con el mapa de `actionEffects.ts:115-127` $\Rightarrow$ esos asesores **jamás** aplican su único bonus mecánico posible.
- **Estado "inactivo" es ficción:** ningún engine inactiva asesores; la UI tiene renders para una rotación que no existe. Tampoco existe `calculateAdvisorEffects` ni reactivación por turno que el doc de diseño describe.

7. Sistemas muertos, redundantes y contradicciones (inventario completo en Sección 4)

- **36 contradicciones UI/texto vs comportamiento** catalogadas (C1-C36): tooltips que prometen efectos que no ocurren (53 acciones), warnings que describen castigos inexistentes (`jugada\_audaz`), chips de tendencia permanentes "—", "Riesgo derrota: medio" en rangos que son victoria (45-49%), badges de efectos sin traducir, estabilidad mostrada como "%" cuando es 0-100, pasiva sindicalista que dice "+1 acción por apoyo popular" siendo incondicional, etc.
- **Inventario de muertos/decorativos:** `satisfiesDemand` (44 acciones), `eventWeights.ts` entero, `actionCategories.ts` zombie (autorreconocido), matcher textual, `getMaxLoans` sin cablear, `baseActionsModifier`/`ironman` de dificultad, pesos de eventos, `requiredAdvisors` (ningún evento lo usa), código de carrera completo (reservado), `impeachment`/`institutional\_coup`, chips de tendencia, doble piso de popularidad que blindo contra caídas.
- **Corrección a una creencia previa:** `legislativeSupport` NO es siempre null — se asigna en legislativas de año 2 y persiste hasta la próxima elección; es "null durante el año 1 de cada mandato".
- **Dificultades no-normal:** todo el balance de hard/legend (decay $\times 1.3/\times 1.5$, ingreso $\times 0.85/\times 0.7$, sin préstamos) es inerte porque no existe selector de dificultad en UI.

Índice

- **Sección 1** — Variables de estado y resultados del juego (fórmulas, resets, umbrales)
- **Sección 2** — Acciones políticas y grupos de interés (18 grupos, antagonismo, demandas, la cuenta de "demasiado fácil")
- **Sección 3** — Sistema ideológico y asesores (perfil, ejes, campo $\times$ consumidor)
- **Sección 4** — Sistemas muertos, redundantes y 36 contradicciones UI vs código

1. Variables de estado y resultados del juego

Radiografía EXACTA (solo lectura) del juego **GobernArg** (TypeScript, React, MVP presidente-only). Repo auditado: `project-bolt-sb1-dzggso GobernArg 21-11/project`. Todas las referencias son `archivo:función:línea` aproximadas al código leído. **No se propone ningún cambio ni balance:** solo se documenta el sistema real con números exactos.

Archivos clave: `src/types/game.ts`, `src/engine/` (gameEngine, engineShared, turnProcessor, electionEngine, eventResolver, archetypeEngine, difficultyEngine, legitimacyEngine, axisEngine, groupAgendaEngine), `src/utls/` (popularidad, electionSystem, victoryConditions, actionEffects, actionCalculator, interactionCosts, crossGroupEffects), `src/data/` (careerRules, specialAbilities, midtermStrategies, groupAntagonists, interestGroups, defeatReasons, calendar).

PARTE 1 — VARIABLES DE ESTADO

1.1 `popularity` (types/game.ts:461)

Aspecto	Detalle exacto
Valor inicial	50 base (gameEngine.ts:getInitialGameState:84) → en partida real: ARCHETYPE_STARTING_POPULARITY[archetype] = político 50, empresario 50, sindicalista 50, comunicador 70 (engineShared.ts:36-41, aplicado en createNewGame:218)
Rango	Clamp [0,100] al recalcular (popularidad.ts:15); en turno se aplica Math.max(0, ...) (turnProcessor.ts:453) y eventos claman (eventResolver.ts:395)
Cómo sube	Efectos inmediatos de acciones: popularityChange × 0.40 (POPULARITY_GLOBAL_FACTOR) × archetypeMultiplier × advisorMultiplier × strategyMultiplier × reunionMultiplier × diminishingFactor × axisEffectiveness (actionEffects.ts:43,72); objetivos completados (+10/+15/+20 según cargo, victoryConditions.ts getPositionObjectives); satisfacer demanda manual +1 (gameEngine.ts:satisfyGroupDemand:523) y por acción cumplida en deadline +10 al grupo (no a pop); pasivas de estrategia midterm: abrirse +1/turno (midtermStrategies.ts:23); habilidades especiales: +8 a +15 (specialAbilities.ts); ingreso de eventos con choices
Cómo baja	Desgaste natural por cargo: presidente 10/turno × difficulty.popularityDecayMultiplier (0.5/1.0/1.3/1.5) (turnProcessor.ts:444-453); eventos (con resilience ×(1-0.30) del comunicador, eventResolver.ts:390); inflación: -5/turno a 3-4 emisiones, -20/turno a 5+ (turnProcessor.ts:processInflation:231); mantenimiento diferido de acciones grandes -2 (actionEffects.ts:214); acciones impopulares (factor 0.40)
Consecuencias	Umbral de derrota DEFEAT_POP_THRESHOLD: presidente 30 (engineShared.ts:52); 2 turnos consecutivos → low_popularity (victoryConditions.ts:9,52); <10 + estabilidad <20 → impeachment; elección (35% del peso); +1 acción base si ≥75 (actionCalculator.ts:31); victoria final requiere ≥60 (y objetivo de 80)
Referencia	types/game.ts:461; utls/popularidad.ts:3; engine/turnProcessor.ts:158 (checkDefeat), 444 (decay)

1.2 `popularidadGrupos` / `popularidadPolitica` (types/game.ts:462-463)

Aspecto	Detalle
Valor inicial	50/50 (getInitialGameState:85-86)
Cómputo	popularidadGrupos = promedio simple de groupRelations (popularidad.ts:21-26); popularidadPolitica = popularity cruda (espejo); popularidadTotal = max(popularity, round(grupos×0.4 + política×0.6)) clamp [0,100] (popularidad.ts:11-15) — los grupos solo suben popularidad, nunca la bajan
Consecuencias	Se escriben en recalState (engineShared.ts:110-114) pero ningún componente React las consume (grep solo engine/types) — informativas
Referencia	utls/popularidad.ts:3; engine/engineShared.ts:recalcState

1.3 `budget` (types/game.ts:464)

Aspecto	Detalle		
Valor inicial	POSITION_STARTING_BUDGET[presidente]=3500 + 500 si empresario (engineShared.ts:30-34; gameEngine.ts:219). Dificultad no modifica el inicial		
Rango	Sin techo; piso efectivo: derrota por <0 dos turnos consecutivos; clamp inferior solo en penalizaciones Math.max(0,...) puntuales (aplicarLegislativeOutcome)		
Cómo sube	Ingreso fiscal por cargo: presidente +500/turno (intendente 200, gobernador 350; engineShared.ts:18-22) × (1-debtServiceRatio) × (1+incomeModifier activo) × (1+archetypeIncomeBonus 0.20 empresario) × difficulty.incomeMultiplier (1.2/1.0/0.85/0.7) (turnProcessor.ts:388-411); acciones con budgetChange positivo × diminishingFactor × costMultiplier × axisCostMultiplier (actionEffects.ts:73); infraestructura da pendingEffect +10% ingresos × 3 turnos (actionEffects.ts:220); préstamos/emitir dinero +150; habilidades +400/+500; recompensas de objetivos (+500/+1000/+2000); reelección: 3500 + 10% del budget (electionEngine.ts:101)		

Aspecto	Detalle		
Cómo baja	Costos de acciones (budgetChange negativo × factores); gasto fijo de gobierno: presidente −350/turno (engineShared.ts:24-28); mantenimiento diferido: \	budgetChange\	≥200 → 15% del costo en 2-4 turnos + (−2 pop) (actionEffects.ts:207-216); servicio de deuda: cada préstamo debtCount+1 (cap 3 hardcodeado, turnProcessor.ts:366) → ratio = debtCount×0.10 → ingreso ×(1−ratio) (turnProcessor.ts:390-391); interacciones: reunión −10, negociar −25×influencia, conceder −45×influencia (interactionCosts.ts:8-11); inflación: −50/turno (3-4 emisiones), −300/turno (5+); eventos; concesiones cruzadas no cuestan budget
Consecuencias	<0 × 2 turnos consecutivos → negative_budget; budgetImpact electoral = 50 + growthRate/2 clamp [0,100] con growth contra historicalBudget[0] (electionSystem.ts:141-149); victoria final requiere budget > 0 y objetivo de 10000; prerequisites minBudget por acción		
Referencia	engine/turnProcessor.ts:388-417; utils/actionEffects.ts; engine/electionEngine.ts:101		

1.4 `votingIntention` (types/game.ts:481) — "intención de voto"

Aspecto	Detalle
Valor inicial	50 (getInitialGameState:104), recalculado por recalState (engineShared.ts:120) con la fórmula de electionSystem.ts
Fórmula	VI = popImpact×0.35 + budgetImpact×0.15 + groupsSupport×0.15 + objectivesImpact×0.15 + stability×0.05 + activity×0.15; si _archetypeElectionRetention>0 (político 0.10): VI += retention×(100−VI); clamp [0,100] (electionSystem.ts:108-133). Para opción: (VI + PROMOTION_DIFFICULTY[opción]) × ascensionMultiplier, reelección +5 (electionSystem.ts:79; careerRules.ts:20-24)
Cómo sube/baja	No se modifica directamente salvo por acciones con multiEffects.votingIntentionChange (ej. emitir_dinero −2, × diminishingFactor, turnProcessor.ts:337) y recalculo por cambios en sus componentes
Consecuencias	Gana elección si votesPercentage ≥ 45 (MIN_VOTES_TO_WIN, electionSystem.ts:21,88); al resolver: votingIntention = votesPercentage (electionEngine.ts:50). No participa en la victoria final ni en derrotas
Referencia	utils/electionSystem.ts:17 (ELECTION_CONSTANTS), 108

1.5 `stability` (types/game.ts:492)

Aspecto	Detalle
Valor inicial	50 (getInitialGameState:115); no depende de arquetipo/dificultad
Rango	Clamp [0,100] (turnProcessor.ts:335, eventResolver.ts:399)
Cómo sube	multiEffects.stabilityChange de acciones (× diminishingFactor); resultado legislativo: landslide +10, clear +5 (eventResolver.ts:112-116); estrategias midterm (año ≥3): negociar +3, abrirse +5/turno (midtermStrategies.ts); eje cerrado ≤−80: +5/turno (axisEngine.ts:63); habilidades +5/+10
Cómo baja	multiEffects negativos (emitir_dinero −3); legislativo: tie −3, minority −8, defeat −15; estrategias: acelerar −3, jugada_audaz −5/turno; eje convocante ≥80: −5/turno; eventos (con resilience del comunicador); habilidades −3/−8
Consecuencias	<25 notifica inestabilidad (turnProcessor.ts:54); impeachment si pop<10 + stab<20 ×2 turnos; 5% del peso electoral; advertencia de crisis en eventos
Referencia	engine/turnProcessor.ts:335,420-441; engine/axisEngine.ts:43

1.6 `legitimacy` (types/game.ts:502)

Aspecto	Detalle		
Valor inicial	60 (getInitialGameState:125)		
Rango	Clamp [0,100] (turnProcessor.ts:336; gameEngine useSpecialAbility:449)		
Cómo sube	multiEffects.legitimacyChange × diminishingFactor; acciones cultura/diplomacia +3 por calculateLegitimacyChange (legitimacyEngine.ts:9); habilidades +5/+8		
Cómo baja	emitir_dinero −5; decretos forzados (pop≤−10 y budget≤−400) −8; medidas impopulares −min(5, \	pop\	×0.1); habilidades −5
Consecuencias	prerequisites.minLegitimacy para desbloqueo de acciones; checkLegitimacyCostMultiplier (legitimidad ≤0 → costo ×2) existe pero no es consumida por nadie (grep sin usos). Sin derrota asociada		
Referencia	engine/legitimacyEngine.ts:4,27; engine/turnProcessor.ts:370-371		

1.7 `groupRelations` (types/game.ts:490) — "apoyo de grupos"

Aspecto	Detalle
Valor inicial	Por subgrupo = subgroup.baseSupport (interestGroups.ts): empresarios 40, sector-agricola 45, sector-financiero 35, sindicatos 50, clase-media 55, sectores-populares 60, clase-alta 30, minorias-etnicas 40, ongs 45, ambientalistas 45, feministas 50, estudiantiles 55, cooperativas 45, aliados 70, opositores 20, artistas 45, deportistas 50, academicos 45 (gameEngine.ts:67-72/190-195)
Rango	Clamp [0,100] en casi todas las vías (gameEngine.ts:310-313; crossGroupEffects.ts:22; eventResolver.ts:402; turnProcessor.ts:340)

Aspecto	Detalle
Cómo sube	Efectos grupales de acciones: explícitos (explicitGroupEffects) o matcher textual popularityChange × influence/10 (actionEffects.ts:144-170), ×1.1 si bono de reunión activo; interacciones: reunión +2, negociar +4, conceder +15 (interactionCosts.ts:14-18); satisfacer demanda +5 manual / +10 por cumplir acción en deadline (gameEngine.ts:495; groupAgendaEngine.ts:158-162); habilidades (+3 a +15); eje convocante ≥80: +10/turno a todos; eventos
Cómo baja	Antagonismo cruzado: cuando un grupo gana apoyo, sus antagonistas pierden round(ganancia × ratio) (crossGroupEffects.ts:9-26; matriz groupAntagonists.ts:10-43, ratios 0.15-0.6); conceder: todos los demás grupos −max(2, round(influencia×0.5)) (gameEngine.ts:316-322); demanda ignorada en deadline: −round(influencia) (groupAgendaEngine.ts:170-174); satisfacer demanda descuenta antagonistas según matriz (gameEngine.ts:503-513); estrategia abrirse: aliados −2/turno; eje cerrado ≤−80: −10/turno a todos
Consecuencias	Promedio ponderado por influencia = 15% del voto electoral (electionSystem.ts:151-172); promedio simple alimenta popularidadGrupos; moods de grupos (≥70 contento, ≥50 neutral, ≥35 disconforme/enojado, <35 enojado/radicalizado, groupAgendaEngine.ts:132-147); objetivo "Estabilidad Total" exige 5 grupos en 75-85; apoyo legislativo usa promedio simple (eventResolver.ts:44-47)
Referencia	engine/gameEngine.ts:applyInteraction:248; utils/crossGroupEffects.ts

1.8 `legislativeSupport` (types/game.ts:486)

Aspecto	Detalle
Valor inicial	null (getInitialGameState:109) → (??100) en chequeos
Cómputo	Solo en midterms (año 2 turno 4, calendar.ts:29-31): legislativeSupport = clamp(officialismVotes × 1.1, 25, 75) (eventResolver.ts:74-75); officialismVotes = 35 + (avgPop−50)×0.25 + (avgGrupos−50)×0.15 + objetivos×10%×... + (stab−50)×0.1 + ruido ±3 clamp [28,58] (eventResolver.ts:36-68)
Efectos	Resultado ajusta estabilidad (+10/+5−3/−8/−15); consecuencias por turno si <35 (45% evento oposición), <38 (25%), >45 (25% sobreconfianza) (eventResolver.ts:164-212); desbloquea estrategias midterm: acelerar si >42, abrirse si ≥3 grupos >50 (engineShared.ts:128-146); prerequisite minLegislativeSupport de acciones
Consecuencia crítica	Umbral del golpe: stab<10 y legislativeSupport<25 — el clamp mínimo es 25 → condición estructuralmente imposible (ver Hallazgos)
Referencia	engine/eventResolver.ts:36-129

1.9 Variables derivadas / acumuladores

Variable	Inicial	Mecánica exacta	Referencia
moneyPrintingCount	0	+1 por ejecución de emitir_dinero (cooldown 3, diminishingFactor 0.65); reset a 0 en cada reelección (electionEngine.ts:114); 3-4 emisiones: −5 pop/−50 budget por turno; 5-6: −20/−300; ≥7 → derrota hyperinflation	turnProcessor.ts:357-358, processInflation:231; victoryConditions.ts:76
debtCount / debtServiceRatio	0 / 0	debtCount = min(3, debtCount+1) por acción isLoan; ratio = debtCount × 0.10; ingreso × (1−ratio); reset en reelección (electionEngine.ts:122-123)	turnProcessor.ts:365-368
consecutiveLowPopularity	0	+1/turno si pop < umbral cargo (presidente 30), reset si no; derrota al llegar a 2	turnProcessor.ts:checkDefeat:158-169
consecutiveNegativeBudget	0	+1/turno si budget < 0; derrota al 2	turnProcessor.ts:171-175
impeachmentConsecutiveTurns	0	+1/turno si pop<10 y stab<20; derrota al 2	turnProcessor.ts:181-185; victoryConditions.ts:62
coupConsecutiveTurns	0	+1/turno si stab<10 y legislativeSupport<25; derrota al 3	turnProcessor.ts:186-190; victoryConditions.ts:69
actions / baseActions	5 / 5 base	Recalculado: base por cargo (presidente 1) +1 político +1 empresario + pasiva sindicalista +1 + asesores activos (bonusActions) +1 si pop ≥75 (actionCalculator.ts:3-36); pasivas lo re-setean cada inicio de turno	engineShared.ts recalcState:110; turnProcessor.ts:312
Ejes (radicalConciliadorAxis, populistaTecnicoAxis, cerradoConvocanteAxis)	0/0/0	Shift por categoría de acción (axisEngine.ts:9-19) + shift por pasiva cada turno (político +2/+1, empresario +2/−1, sindicalista −2/−2, comunicador +2; archetypeEngine.ts:55-70); clamp [−100,100]; extremos ±80 aplican modificadores (axisEngine.ts:43-70)	engine/axisEngine.ts
groupAgendas / groupMoods	[] / todos 'neutral'	Máx 2 demandas activas; prob base 0.08 +0.05×ignoredTurns (tope 0.25); deadline 3-5 turnos; negociada: deadline +4; penalización −influencia al vencer	groupAgendaEngine.ts:15-181
concessionsThisTerm	0	Máx 4 concesiones/mandato; requiere ≥1 reunión/negociación previa	gameEngine.ts:261-271
temporarySupportBonuses	{}	Reunión: bono ×1.1 efectos de acciones del grupo por 3 turnos (solo actionMultiplier se consume; el bonus numérico guardado nunca se aplica)	interactionCosts.ts:21-24; actionEffects.ts:177
Acumuladores _archetype*	—	incomeBonus 0.20 (empresario), electionRetention 0.10 (político), extraActions +1 (sindicalista), extraLoans +1 (empresario, no consumido, ver Hallazgos), eventResilience 0.30 (comunicador)	specialAbilities.ts:175-196; archetypeEngine.ts:6

No existen como variables de estado: `socialConflict` (lo más cercano son `groupMoods`/`stability`), `income` (es constante por cargo `POSITION\_INCOME`, no un estado), ni `voteIntention` (es `votingIntention`).

PARTE 2 — RESULTADO DEL JUEGO

2.1 Vías de derrota (todas evaluadas en `checkAllDefeatConditions`, victoryConditions.ts:48, llamado desde `checkDefeat`, turnProcessor.ts:193, al final de cada `processEndTurn` paso 9)

#	Razón (`DefeatReason`)	Condición exacta	Contador	¿Alcanzable?
1	low_popularity	popularity < 30 (presidente; 25 gobernador, 20 intendente) Y consecutiveLowPopularity ≥ 2	+1/turno bajo umbral, reset al salir (turnProcessor.ts:164-169)	Sí
2	negative_budget	budget < 0 Y consecutiveNegativeBudget ≥ 2	+1/turno en negativo	Sí
3	impeachment	popularity < 10 Y stability < 20 Y impeachmentConsecutiveTurns ≥ 2	+1/turno (turnProcessor.ts:181-185)	Sí en teoría, no en la práctica: pop<10 ⊆ pop<30, ambos contadores suben juntos y low_popularity se chequea primero (victoryConditions.ts:52-54) → el reason siempre será low_popularity
4	institutional_coup	stability < 10 Y legislativeSupport < 25 Y coupConsecutiveTurns ≥ 3	+1/turno (turnProcessor.ts:186-190)	NO: legislativeSupport = clamp(..., 25, 75) (eventResolver.ts:75) nunca es <25; antes de midterms es null → ??100
5	hyperinflation	moneyPrintingCount ≥ 7	Directo, sin contador (victoryConditions.ts:76)	Sí: emitir_dinero tiene cooldown 3, 16 turnos/mandato permiten ≥5 emisiones por mandato; se resetea en reelección, así que exige 7 en un mismo mandato
6	election_loss	votesPercentage < 45 al resolver elección pendiente	En resolvePendingElection (electionEngine.ts:57-62): gameOver=true, victorious=false, defeatReason='election_loss'	Sí

Al perder: `gameOver=true`, `victorious=false`, `defeatReason` seteado (turnProcessor.ts:194-198 / electionEngine.ts:58-61). Textos UI en `data/defeatReasons.ts`.

2.2 Elección de re-elección

Disparo: fin de mandato = `year===4 && turn===4` (turnProcessor.ts:512). Si `term < MAX\_TERMS[presidente]=2` (careerRules.ts:12-16) hay opción `reelection` (electionSystem.ts:32-55); si no quedan opciones → `finalizePresidentialCareer` directo (turnProcessor.ts:519-523).

Fórmula de votos (`processElectionResultsForOption`, electionSystem.ts:83-101 → `calculateVotingIntentionForOption`:62-81):

```
...  
  
base = calculateVotingIntention(state) // electionSystem.ts:108  
votes = clamp[(base + PROMOTION_DIFFICULTY['reelection']+=5) × 1.0, 0, 100]  
victory = votes >= 45 // MIN_VOTES_TO_WIN  
...
```

Componentes de `base` (pesos en ELECTION_CONSTANTS.WEIGHTS, electionSystem.ts:22-29), todos en escala 0-100:

Componente	Peso	Fórmula literal
popularityImpact	0.35	Promedio de historicalPopularity.slice(-4) (últimos 4 trimestres, electionSystem.ts:135-139)
budgetImpact	0.15	50 + ((budget – historicalBudget[0]) / historicalBudget[0]) × 100 / 2, clamp [0,100] (electionSystem.ts:141-149)
groupsSupport	0.15	Σ(support×influencia)/Σinfluencia sobre todos los subgrupos (electionSystem.ts:151-172)
objectivesImpact	0.15	completedObjectives.length / objectives.length × 100 (electionSystem.ts:174-179)
stabilityBonus	0.05	stability directo (electionSystem.ts:181-183)
activityImpact	0.15	min(100, accionesDelMandatoActual / 16 × 100) — turnLog filtrado por position+term (electionSystem.ts:191-207)
Retención pasiva	—	político: VI += 0.10 × (100 – VI) (electionSystem.ts:127-130)

Al perder (electionEngine.ts:57-62): `gameOver=true`, victorious=false, defeatReason='election_loss'; `electionResults` guardado con `details` de cada componente.

Al ganar — reset campo por campo (`resolvePendingElection`, electionEngine.ts:64-137):

- `electionResults` seteado; `votingIntention = votesPercentage`; `pendingElection=false`; `pendingElectionOptions=[]`
- `term += 1`; `termsByPosition[presidente] += 1`; push de nuevo `CareerMilestone` (victory, type 'reelection')

- `year=1`, `turn=1`; `legislativeResults=null`; `legislativeSupport=null`

- `popularity = round(popularity × 0.7 + 30)` (ej. 80 → 86)

- `budget = POSITION\_STARTING\_BUDGET[presidente]=3500 + round(budget × 0.1)`

- `objectives = getPositionObjectives(presidente)` (los mismos 2); `completedActions=[]`; `pendingEffects=[]`; `interactionHistory={}`; `concessionsThisTerm=0`; `interactionCountByGroup={}`; `lastRandomEventTurn=0`; `randomEventsThisTerm=0`; `lastEventFiredTurns={}`; `advisors=[]`; `advisorActionUsed=false`; `moneyPrintingCount=0`; `consecutiveLowPopularity=0`; `consecutiveNegativeBudget=0`; `groupAgendas=[]`; `groupMoods=[]`; `actionUsageCount={}`; `actionCooldowns={}`; `debtCount=0`; `debtServiceRatio=0`; `completedObjectives=[]`; `midtermStrategy=null`; `pendingMidtermStrategy=false`; `availableMidtermStrategies=[]`; `audazTurnsCount=0`; `abilityCooldowns={}`; `impeachmentConsecutiveTurns=0`; `coupConsecutiveTurns=0`; `historicalBudget=[budget]` (para el budgetImpact)

- **NO se resetean:** `stability`, `legitimacy`, `votingIntention`, ejes, `groupRelations`, `historicalPopularity` (sigue acumulando toda la carrera), `defeatReason`

- Cierre con `recalcState` (recalcula popularity por popularidad.ts, acciones y votingIntention)

2.3 Victoria final

Objetivos del presidente (`getPositionObjectives('presidente')`, victoryConditions.ts:228-263) — fijos, sin rotación ni aleatoriedad, idénticos en cada mandato:

Objetivo	Requisitos (AND)	Recompensa
national-prosperity "Prosperidad Nacional"	popularity ≥ 80 Y budget ≥ 10000	+20 popularity
total-stability "Estabilidad Total"	groupSupport (AND de 5): empresarios ≥85, sindicatos ≥85, clase-media ≥80, clase-alta ≥75, sectores-populares ≥80	+2000 budget

Evaluación de objetivos (`updateObjectives`, victoryConditions.ts:83-148): cada requisito es un check independiente y se combinan con **AND** (`checks.every`); `progress` = promedio de progresos. Corre en `processEndTurn` paso 10 (turnProcessor.ts:575) y dentro de `finalizePresidentialCareer`. Recompensas aplicadas una sola vez (`applyObjectiveRewards`, turnProcessor.ts:207-215).

Condición de victoria (`checkVictoryConditions`, victoryConditions.ts:13-19):

```
...  
  
victorious = objectives.every(o => o.completed) // ambos objetivos  
AND popularity >= 60  
AND budget > 0  
...
```

Dónde y cuándo se chequea: única llamada en `finalizePresidentialCareer` (electionEngine.ts:140-151), que se ejecuta al procesar el turno `year=4, turn=4` del **2º mandato presidencial** (cuando `getAvailableElectionOptions` devuelve vacío, turnProcessor.ts:519-523). Incluye el **re-chequeo:** `updateObjectives(state)` antes de fijar `victorious` (electionEngine.ts:149-150), porque `processEndTurn` evalúa objetivos recién en su paso 10, posterior al paso 5 que invoca al finalizador (fix del commit e8ffca3). El milestone final se registra con `votesPercentage = state.votingIntention` y `result='victory'` (electionEngine.ts:142). `gameOver=true` siempre.

No hay victoria intermedia: `checkVictoryConditions` no se invoca en ningún otro lado; cumplir objetivos en el mandato 1 no gana nada (solo recompensas).

2.4 Condiciones intermedias (impeachment, golpe, hiperinflación, crisis)

- **Impeachment:** ejecutable (código real, victoryConditions.ts:62-66) pero efectivamente ensombrecido por `low\_popularity` (mismo timing, chequeo previo). Umbral: pop<10 + stab<20 × 2 turnos consecutivos.
- **Golpe institucional:** código ejecutable (victoryConditions.ts:69-73) pero **inalcanzable:** `legislativeSupport` nunca <25 por clamp (eventResolver.ts:75) y vale `null→100` antes de las midterms.
- **Hiperinflación:** alcanzable; 7 emisiones en un mismo mandato (el reset por reelección impide acumular entre mandatos).
- **Crisis inflacionaria intermedia** (no derrota): 3-4 emisiones → −5 pop/−50 budget por turno; 5+ → −20/−300 por turno (processInflation, turnProcessor.ts:231-259). Crisis de eventos: probabilidad × crisisProbabilityMultiplier de dificultad (0.3/1.0/1.5/2.0), máx 1 evento aleatorio por turno, cooldown global 3 turnos, máx 5 por mandato (eventResolver.ts:227-228).

Hallazgos de consistencia (solo señalados, sin propuestas)

1. **`institutional\_coup` es inalcanzable:** el clamp ``Math.max(25, ...)`` de ``legislativeSupport`` (eventResolver.ts:75) contradice el umbral `< 25`` (victoryConditions.ts:69). ``coupConsecutiveTurns`` nunca puede incrementarse. El tipo ``DefeatReason`` y toda la config UI del golpe son código muerto.
2. **`impeachment` prácticamente inalcanzable como razón:** `pop<10` implica `pop<30`, ambos contadores suben en los mismos turnos y ``low_popularity`` se evalúa primero en ``checkAllDefeatConditions``; el ``reason`` registrado siempre será ``low_popularity``.
3. **`extraLoans` (empresario) es pasiva muerta:** ``getMaxLoans`` (archetypeEngine.ts:87) no es llamada por nadie; `turnProcessor.ts:366` usa el literal ``Math.min(3, ...)``. El texto "máx 4 préstamos" no se cumple en el código.
4. **`checkLegitimacyCostMultiplier` ("legitimidad 0 ⇒ acciones ×2") no es consumida** por ningún archivo: la legitimidad en 0 no tiene efecto mecánico; la variable solo pesa en ``prerequisites.minLegitimacy``.
5. **El `bonus` numérico de `temporarySupportBonuses` nunca se aplica** a ``groupRelations``; solo ``actionMultiplier`` (×1.1) es leído (actionEffects.ts:177-190). La reunión da +2 una sola vez vía ``supportGain``.
6. **`popularidadGrupos`/`popularidadPolitica` se calculan en cada `recalcState` pero ningún componente las muestra** (grep: solo engine/types). Además la fórmula ``max(política, 0.4×grupos+0.6×política)`` hace que el apoyo grupal solo pueda *subir* la popularidad.
7. **Reelección: `historicalPopularity` no se resetea** (sí ``historicalBudget``, electionEngine.ts:135): el ``popularityImpact`` electoral mezcla datos de ambos mandatos solo si el slice(-4) cae en el borde; coherente, pero asimétrico con budget.
8. **Asesores eliminados en cada reelección** (``advisors=[]``) sin aviso en el flujo de elección; estabilidad, legitimidad, ejes y ``groupRelations`` tampoco se resetean (asimetría de resets: se borra ``moneyPrintingCount`` pero no ``stability``).
9. **`canRunForOption` (minPop por opción) no se aplica en `resolvePendingElection`:** la única puerta real es el cálculo de votos; reelección tiene minPop=0 de todos modos.
10. **Reset de popularidad post-reelección puede subirla:** ``pop×0.7+30`` mapea 100→100, 80→86, 50→65; un presidente impopular (40) queda en 58.
11. **Desgaste vs. dificultad:** solo ``popularityDecayMultiplier`` e ``incomeMultiplier``/``crisisProbabilityMultiplier``/``loansAvailable`` se consumen; ``baseActionsModifier`` (−1 en hard/legend) y ``ironman`` están declarados "sin consumir" (difficultyEngine.ts:10-12) — la dificultad no cambia los puntos de acción pese al campo.
12. **Warning de estabilidad:** el mensaje de inestabilidad dispara a `<25`` (turnProcessor.ts:54) pero no hay derrota por estabilidad sola; solo actúa como co-condición de impeachment.
13. **`processInflation` es excluyente (if/else):** a 5+ emisiones aplica −20/−300 y **ya no** el −5/−50 menor; el texto de "presión inflacionaria" solo aparece en count===3.
14. **Presupuesto inicial vs. descripción de arquetipo:** comunicador arranca con 70 de popularidad pero el desgaste de presidente (10/turno) lo devuelve a la media en ~2-3 turnos si no actúa.
15. **Estrategia `jugada\_audaz`:** el texto promete "penalización fuerte" tras 2 turnos; el código solo revierte a ``negociar`` sin penalidad (turnProcessor.ts:424-437).
16. **Umbral electoral 45 hardcoded** en la UI (ReelectionChoiceModal.tsx:46,93) duplicado respecto a `MIN_VOTES_TO_WIN`` (electionSystem.ts:21) — mismo valor, dos fuentes.
17. **Objetivos inalcanzables presentes:** los objetivos/umbrales de intendente y gobernador y las opciones ``promote-*`` siguen en el código pero son inalcanzables en el MVP (STARTING_POSITION='presidente', careerRules.ts:10); el comentario lo declara explícitamente.
18. **`historicalPopularity` al inicio contiene el valor pre-pasivas** y ``createNewGame`` lo reemplaza (gameEngine.ts:237), consistente; pero el comentario de electionSystem.ts (línea ~242 en gameEngine) contradice la firma actual afirmando que `getInitialGameState` "aplica las pasivas con 'politico'": hoy no lo hace.

Referencias centrales: ``src/types/game.ts:453`` (GameState) · ``src/engine/turnProcessor.ts:265`` (processEndTurn, pipeline de 10 pasos) · ``src/engine/electionEngine.ts:39,140`` · ``src/utls/electionSystem.ts:17`` · ``src/utls/victoryConditions.ts:3,13,48,154`` · ``src/engine/engineShared.ts:18-56`` (constantes de balance).

2. Acciones políticas y grupos de interés

Radiografía de SOLO LECTURA — GobernArg (TypeScript, MVP presidente-only). Branch ``rediseño-visual``. Sin propuestas de cambio ni balance: solo el sistema real, con números literales y referencias archivo:función:línea.

Nota de archivo previa: las rutas asumidas difieren del repo real. ``actionEffects.ts`` vive en ``src/utls/actionEffects.ts`` (no en ``engine/``); ``actionCalculator.ts`` es ``src/utls/actionCalculator.ts`` y calcula **cantidad de acciones por turno**, no efectos; ``interactionCosts.ts`` es ``src/utls/interactionCosts.ts``. **No existe `canExecuteAction`** — la disponibilidad es ``getAvailableActionsForState`` + ``toggleActionSelection`` (ambas en ``src/engine/actionEngine.ts``).

PARTE 1 — ACCIONES

1.1 Tabla-resumen por categoría (61 acciones, `src/data/actionRegistry.ts`)

Valores literales del registry (antes de multiplicadores del motor):

Categoría	N	budgetChange	popularityChange	explicitGroupEffects	cooldown propio	prerequisites	futureEffects	multiEffects
economia	12	-400...+800	-20...+20	2	3	2	4	3
social	10	-400...-200	+12...+20	3	0	0	1	0
infraestructura	14	-700...-80	+2...+20	1	1	4	4	0
diplomacia	7	-300...-150	+8...+15	0	0	1	0	0
seguridad	7	-500...-250	+10...+20	2	1	1	1	1
cultura	8	-400...-200	+10...+18	0	0	0	0	0
educacion	1	-100	+10	0	0	0	0	0
turismo	1	-50	+8	0	0	0	0	0
tecnologia	1	-200	+12	0	0	0	0	0

- **explicitGroupEffects (8 acciones):** `subsidios\_industriales` (empresarios +8, sindicatos +8, sector-financiero -9, ongs -6), `fomento\_emprendimiento` (empresarios +9.6, cooperativas +6, clase-media +8.4), `plan\_viviendas` (sectores-populares +12), `cobertura\_social` (sectores-populares +12), `tercera\_edad` (sectores-populares +9, clase-media +10.5, ongs +9), `viviendas\_rurales` (sectores-populares +9), `seguridad\_ciudadana` (clase-media +14), `sistema\_vigilancia` (clase-media +10.5).
- **Las otras 53 declaran `affectedGroups` SIN efecto real** (ver 1.3 y Hallazgos).
- **Cooldown explícito (5):** emitir_dinero:3, prestamo_internacional:8, prestamo_local:8, estudio_factibilidad:2, seguridad_ciudadana:3.
- **minBudget:** todas lo tienen; valores posibles 0,50,80,100,150,200,250,300,350,400,450,500,600,700 (siempre = |budgetChange| salvo excepciones: emitir_dinero minBudget:0 con +150).
- **minLegislativeSupport (1):** reforma_impositiva:45 (con comentario en código: imposible — legislativeSupport es null hasta año 2 y fijo después; se mantiene deliberado).
- **minGroupSupport (1):** tratado_comercio: empresarios≥60.
- **requiredActions (7):** reforma_impositiva←mejorar_recaudacion; prestamo_internacional←mejorar_recaudacion; energia_renovable, construccion_hospitales, infraestructura_vial←estudio_factibilidad; modernizacion_aeropuertos←estudio_factibilidad+infraestructura_vial; lucha_narcotrafico←fortalecimiento_justicia.
- **satisfiesDemand:** 44 acciones lo declaran, pero **ningún código del engine lo lee** (solo types/tests). Campo muerto.

1.2 Cadena de ejecución completa

Selección (pre-turno): `toggleActionSelection` (actionEngine.ts:97) → verifica `getAvailableActionsForState` (actionEngine.ts:28), que filtra por: exclusión por cargo (vacía, engineShared.ts:46), préstamos según dificultad (loansAvailable false en difícil/extremo, difficultyEngine.ts:44,52), `availableForPositions`, cooldown>0, prerequisites (requiredActions/minLegislativeSupport/minLegitimacy/minGroupSupport), budget≥minBudget, minPopularity, advisorRequired, groupSupportRequired, y `unlockedActions` (arranca con TODAS, gameEngine.ts:118). Cada acción mapea a `actionCost`: reforma = 1 + penalidad legislativa; no-reforma = 1.

Penalidad legislativa (`getLegislativePenalty`, actionEngine.ts:16): legislativeSupport null → 0; ≥45 → -1; ≥38 → 0; ≥35 → +1; <35 → +2. Reforma = categoría economia/infraestructura o |budgetChange|≥200 (actionEngine.ts:12-13) → casi todas; excepciones no-reforma: emitir_dinero, estudio_factibilidad, fomentar_turismo, promover_educacion.

Fin de turno (`processEndTurn`, turnProcessor.ts:265), orden exacto:

1. `processCalendarEvents` → `resolveLegislativeConsequences` → `applyArchetypePassives` (pasos 0–0.6).
2. **Paso 1 — por cada acción seleccionada** (turnProcessor.ts:323-373): `calculateActionEffects` (actionEffects.ts:22) y aplica:
 - budget += budgetChange; popularity += popularityChange; stability/legitimacy/votingIntention (multiEffects ×diminishingFactor).
 - **Grupos:** `groupRelations[ge.groupId] += ge.supportChange` (clamped 0-100) — **crudo, sin 0.40 ni diminishingFactor**.
 - push pendingEffects; completedActions; emitir_dinero → moneyPrintingCount++; actionUsageCount++; `actionCooldowns[id] = cooldown ?? getDefaultCooldown`; préstamos → debtCount=min(3, +1), debtServiceRatio=debtCount×0.10; legitimidad vía `calculateLegitimacyChange` (legitimacyEngine.ts:4: +3 si cultura/diplomacia; -8 si pop≤-10 y budget≤-400; -min(5, |pop|×0.1) si pop<0); shift de ejes.
3. **Paso 1.5 — antagonismo cruzado** (turnProcessor.ts:380): `applyCrossGroupEffects(state, groupChanges)` — **una sola pasada** sobre los cambios ya calculados (fix Punto 13).
4. **Paso 2 — pendingEffects** que activan este turno (processPendingEffects, actionEffects.ts:267).

5. **Paso 3 — ingreso/gastos fijos:** presidente income 500, maintenance 350 (engineShared.ts:18-28), $\times(1-\text{debtServiceRatio})$, $\times(1+\text{incomeModifier activo})$, $\times(1+ \text{archetypeIncomeBonus})$, $\times\text{difficultyIncomeMultiplier}$.
6. Pasos 3.5-4: estrategia midterm; **desgaste de popularidad –10/turno presidente** ($\times\text{difficultyDecay}$); efectos de ejes extremos (± 80).
7. Pasos 7.x: cooldowns de interacciones (turnsLeft 2), cooldowns de acciones, inflación por emisión, moods/agendas/negociaciones/penalizaciones de demanda, cooldowns de habilidades.
8. ``recalcState`` → ``checkDefeat`` → notificaciones → objetivos.

Fórmula de efectos inmediatos (actionEffects.ts:71-84):

...

$\text{popularityChange} = \text{effPop} \times \text{archetypeMult} \times \text{advisorMult} \times \text{strategyMult} \times \text{reunionMult} \times 0.40 \times \text{diminishing} \times \text{axisEff}$

$\text{budgetChange} = \text{budgetChange} \times \text{diminishing} \times \text{costMultiplier} \times \text{axisCost}$

donde $\text{diminishing} = (\text{action.diminishingFactor} ?? 0.80)^{\text{usageCount}}$

y si $\text{usageCount} \geq 5$ y $\text{pop} > 0$ → la popularidad se INVIERTE a negativa

$\text{groupEffects} = \text{crudos}$ (solo $\times\text{bonus}$ temporal de reunión si aplica, $\times 1.1$)

...

Multiplicadores: arquetipo (politico $\times 1.2$ diplomacia; empresario $\times 1.3$ economía/ $\times 0.9$ resto; sindicalista $\times 1.3$ social/ $\times 0.9$; comunicador $\times 1.1$), asesor activo de especialidad matching $\times 1.2$ por asesor, estrategia midterm (año ≥ 3), reunión $\times 1.1$, ejes extremos.

1.3 Matcher textual — estado: MUERTO

``calculateGroupEffects`` (actionEffects.ts:144): si hay `explicitGroupEffects` → esos y **solo esos**; si no, matchea ``subgroup.interests`` contra ``action.description`` (substring case-insensitive) y da ``popularityChange`` $\times (\text{influence}/10)$.

Verificación exhaustiva por script de los 61×36 pares acción/interés: **solo 4 coincidencias** —

Interés	Acción
clase-media: "seguridad"	seguridad_ciudadana, sistema_vigilancia
sectores-populares: "Programas sociales"	cobertura_social
sectores-populares: "vivienda"	plan_viviendas, viviendas_rurales

Las 4 tienen `explicitGroupEffects` → el matcher **nunca se ejecuta en la práctica**. Para las otras 53 acciones: **cero efecto grupal**. "educativa" ≠ "educación", etc.

1.4 Cooldowns

``getDefaultCooldown`` (actionEffects.ts:257): explícito si existe; si no: préstamo/emitir_dinero → 8; $|\text{budget}| > 500$ → 6; ≥ 200 → 3; resto → 1. Se registra en ``state.actionCooldowns`` al ejecutar (turnProcessor.ts:363-364) y decrementa en ``updateActionCooldowns`` (turnProcessor.ts:221): solo se conservan entradas > 1 (decrementadas); $\text{cooldown} = 1$ desaparece al final del mismo turno → **una acción de cooldown implícito 1 se puede usar cada turno**. Bloqueo de selección: ``actionCooldowns[id] > 0`` (actionEngine.ts:40).

1.5 Acciones especiales

emitir_dinero (registry:60-85): +150 budget, popularity +3 (efectivo +1.2 con factor 0.40), stability −3, legitimacy −5, votingIntention −2; cooldown 3; diminishing 0.65 (agresivo); moneyPrintingCount++ → inflación (turnProcessor.ts:231): count=3 → −5 pop y −50 budget **por turno**; count=5 → −20 pop y −300 budget/turno; **count ≥ 7 → derrota hiperinflación** (victoryConditions.ts:76). Avisos a count 3 y 5 (turnProcessor.ts:64-81). Con 16 turnos/mandato y cooldown 3, máximo 5-6 emisiones: la derrota (7) es alcanzable solo en 2º mandato.

Préstamos: internacional +800/legitimacy −8/stability +5 (requiere mejorar_recaudacion, presidente-only); local +500/legitimacy −5. Ambos cooldown 8, diminishing 0.90, futureEffects (−100@t+4 pop−3 / −75@t+3 pop−2). Deuda: ``debtCount`` = $\min(3, +1)$, ``debtServiceRatio`` = $\text{debtCount} \times 0.10$ → ingreso −10/−20/−30 % (turnProcessor.ts:365-368, 390). Límite real: **el hardcodeo ``Math.min(3,...)`` ignora la pasiva de empresario** (``getMaxLoans`` = 3+extraLoans existe en archetypeEngine.ts:87 pero con TODO de integración — no cableado).

Interacciones (``applyInteraction``, gameEngine.ts:248; costos en interactionCosts.ts):

Tipo	Costo \$	Costo acción	Δapoyo	Efectos extra
reunión	10	1	+2	bono temporal 3 turnos: actionMultiplier $\times 1.1$ en acciones que afectan al grupo (pop y apoyo)

Tipo	Costo \$	Costo acción	Δapoyo	Efectos extra
negociar	25×influencia (rango 125-225)	1	+4	genera demanda concreta en 1-2 turnos (deadline +4)
conceder	45×influencia (rango 225-405)	1	+15	pausa demandas 4 turnos; todos los demás grupos −max(2, round(influencia×0.5)); máx 4/mandato; requiere ≥1 reunión o negociación previa en el mandato

Las ganancias por interacción NO disparan antagonismo (no pasan por applyCrossGroupEffects). Pasivas que anulan costos \$: politico gratis con `aliados`; sindicalista gratis con `sindicatos` y `sectores-populares` (specialAbilities.ts:178,187; mecanismo `\_archetypeFreeInteractions`, interactionCosts.ts:63).

PARTE 2 — GRUPOS DE INTERÉS

2.1 Los 18 subgrupos (`src/data/interestGroups.ts`) — valores iniciales exactos

Grupo	Influencia	baseSupport	supportMult	resourceDemand	demandActionIds
empresarios	8	40	1.2	300	reduccion_gasto, incentivos_exportacion, reforma_impositiva
sector-agricola	7	45	1.1	250	subsidios_industriales, viviendas_rurales, infraestructura_vial
sector-financiero	9	35	1.3	400	prestamo_local, control_precios, mejorar_recaudacion
sindicatos	8	50	1.2	350	aumento_salarial, acuerdo_sindical, cobertura_social
clase-media	7	55	1.1	200	programa_educativo, seguridad_ciudadana, transporte_publico
sectores-populares	6	60	1.0	150	plan_viviendas, programa_alimentario, cobertura_social
clase-alta	8	30	1.4	450	reduccion_gasto, reforma_impositiva, atraccion_inversiones
minorias-etnicas	5	40	1.0	100	alfabetizacion, igualdad_genero, plan_viviendas
ongs	6	45	1.1	150	fortalecimiento_justicia, programa_cultural, alfabetizacion
ambientalistas	6	45	1.1	200	reforestacion, energia_renovable, acuerdo_ambiental
feministas	7	50	1.1	250	igualdad_genero, programa_educativo, fortalecimiento_justicia
estudiantiles	5	55	1.0	150	programa_educativo, promover_educacion, red_bibliotecas
cooperativas	5	45	1.0	100	fomento_emprendimiento, subsidios_industriales, plan_viviendas
aliados	8	70	1.3	400	alianza_politica, acuerdo_sindical, participacion_cumbres
opositores	7	20	1.2	300	fortalecimiento_justicia, reduccion_gasto, control_precios
artistas	5	45	1.0	150	programa_cultural, festival_arte, centros_culturales
deportistas	6	50	1.1	200	programa_educativo, infraestructura_vial, transporte_publico
academicos	7	45	1.2	250	promover_educacion, desarrollar_tecnologia, red_bibliotecas

Inicial: `groupRelations[id] = baseSupport` (gameEngine.ts:67-72). `popularity`, `satisfactionLevel`, `lastInteractionEffect` de cada subgrupo no se usan en el engine. **No existe decaimiento por turno del apoyo grupal** — solo sube o baja por acciones/interacciones/demandas/ejes (axis ±10 a TODOS en extremo).

2.2 Demandas/agendas (`src/engine/groupAgendaEngine.ts`)

- **Generación** (`generateGroupAgendas`, :15): máx **2 activas** simultáneas; probabilidad por grupo/turno = min(0.25, 0.08 + ignoredTurns×0.05); respeta pausa por concesión y negociación pendiente.
- **Tipo**: siempre **ID de acción** elegida de `demandActionIds` (Fisher-Yates, filtrando disponibilidad por cargo, prerrequisitos, y no usada en los últimos 3 turnos). La rama legacy de `AGENDA\_TEMPLATES` (texto) es **inalcanzable**: las 7 claves del template tienen todas `demandActionIds`. Deadline: **3-5 turnos** (negociadas: 4).
- **Cumplir ejecutando la acción** (`applyGroupSatisfactionPenalty`, :149): si `completedActions` incluye el id al deadline → **+10 apoyo, +0 popularidad, SIN penalización a antagonistas**.
- **Satisfacer con botón** (`satisfyGroupDemand`, gameEngine.ts:480): **−1 acción, +5 apoyo, +1 popularidad, mood→contento, CON penalización a antagonistas** round(5×ratio).
- **Ignorar**: al deadline, **−influencia** (round; rango −5...−9).
- **Antagonismo al cumplir**: solo en la vía botón (+5). La vía acción (+10) es libre de castigo cruzado.
- Moods** (`updateGroupMoods`, :132): contento ≥70, neutral ≥50, disconforme ≥35, enojado <35, radicalizado <35 con ignoredTurns≥2 / <35+2 turnos. Efecto mecánico del mood: **solo aumenta la probabilidad de demanda** (+0.05/turno ignorado). No hay protestas ni castigos por grupo radicalizado.

2.3 Matriz de antagonismo completa (`src/data/groupAntagonists.ts`)

Ganador → Perdedor	ratio		Ganador → Perdedor	ratio
empresarios → sindicatos	0.5		clase-alta → sindicatos	0.3
empresarios → sectores-populares	0.3		clase-alta → sectores-populares	0.3
sector-financiero → sindicatos	0.5		ambientalistas → sector-agricola	0.4
sector-financiero → sectores-populares	0.3		ambientalistas → sector-financiero	0.2
sector-financiero → ambientalistas	0.15		feministas → clase-alta	0.2
sindicatos → empresarios	0.5		feministas → opositores	0.2
sindicatos → clase-alta	0.4		aliados → opositores	0.6
sectores-populares → empresarios	0.5			
sectores-populares → clase-alta	0.4			

Aplicación: `applyCrossGroupEffects` (crossGroupEffects.ts:9) — solo cuando el cambio es >0, penalización = round(ganancia×ratio), clamp 0-100. Se invoca en **un solo lugar**: paso 1.5 del fin de turno (ganancias por acciones). **No** aplica a interacciones (reunión/negociar/conceder usa su propia regla global) ni a demandas cumplidas por acción (+10). La matriz es **asimétrica** (fin→ambient 0.15 vs ambient→fin 0.2). Solo 8 grupos tienen entradas como agresores; 10 grupos (agricola, clase-media, minorias, ongs, estudiantiles, cooperativas, opositores, artistas, deportistas, academicos) **nunca generan castigo cruzado al ganar**.

2.4 ¿Es demasiado fácil mantener contentos a todos? — La cuenta

Palancas reales que suben apoyo grupal (solo 4 vías):

- 1. 8 acciones con explicitGroupEffects (Δ +6...+14, sin diminishing, sin factor 0.40).
- 2. Interacciones: +2 (reunión, \$10), +4 (negociar, \$125-225), +15 (conceder, \$225-405, con castigo global).
- 3. Demandas: +10 (ejecutar acción) o +5 (botón).
- 4. Satisfy por objetivos/estabilidad: nada. No hay otra.

Recursos por turno (presidente, arquetipo político): acciones = 1 (base presidente, actionCalculator.ts:5-9) +1 (político) = **2**, +1 si popularidad≥75 → 3 típico. Turnos por mandato: 16. Total de slots: ~32-48.

La cuenta, eje a eje:

- **Velocidad neta vs antagonismo (clúster económico):** ciclo de 3 turnos con `subsidiros\_industriales` + `plan\_viviendas`: subsidiros da emp +8, sind +8, fin −9, ongs −6; sus positivos disparan sind −4 (0.5) y emp −4 (0.5). viviendas da sect-pop +12 → emp −6, clase-alta −4.8. **Neto por ciclo: emp −2, sind +4, sect-pop +12, clase-alta −4.8, fin −9, ongs −6**. El antagonismo desgasta empresarios/clase-alta −2/−4.8 por ciclo...
- **...pero una reunión da +2 por turno a CUALQUIER grupo, a \$10, sin antagonismo ni límite.** Con 2-3 acciones/turno, el jugador asigna +2/turno a cada grupo que se desangra: empresarios queda neto 0 con 1 reunión cada 3 turnos. Las ganancias por interacción **no pasan por la matriz** (applyInteraction, gameEngine.ts:309-322, no llama applyCrossGroupEffects).
- **10 de 18 grupos nunca castigan a nadie:** sector-agricola, clase-media, minorias-etnicas, ongs, estudiantiles, cooperativas, opositores, artistas, deportistas, academicos se suben sin costo cruzado alguno. `tercera\_edad` sube 3 de estos gratis (+9/+10.5/+9 en una sola acción).
- **Sin decaimiento:** el apoyo nunca baja solo; llegar a 70 ("contento") es permanente.
- **Demandas como fuente neta positiva:** probabilidad 0.08-0.25/grupo/turno, máx 2 activas; cumplir por acción da **+10 sin castigo a antagonistas** (vs +5 con castigo por botón) — el jugador que ejecuta la acción demandada sale siempre ganando (+10 neto vs −influencia si ignora).
- **Cuello real:** no el antagonismo, sino (a) slots de acción (32-48 por mandato para 18 grupos), (b) dinero (conceder de infl. 8 = \$360 + castigo global), (c) el cooldown de las 8 acciones explícitas. Conceder es la única interacción con costo cruzado y está limitada (4/mandato + requisito previo).

Conclusión con números: sí, el sistema permite mantener contentos a todos. El antagonismo efectivo solo castiga 8 grupos y solo sobre ganancias de 8 acciones; las interacciones (+2/turno/grupo, \$10) son una válvula ilimitada y libre de castigo que contrarresta cualquier fuga (máx castigo típico: −6/turno a empresarios, compensable con 3 reuniones = \$30). El objetivo "Estabilidad Total" (empresarios 85 + sindicatos 85 + sectores-populares 80 + clase-media 80 + clase-alta 75, victoryConditions.ts:249-255) es exigente pero alcanzable precisamente porque `subsidiros\_industriales` sube a los dos antagonistas mutuos (+8/+8) en la misma acción, y la vía demanda (+10) no penaliza.

Hallazgos de consistencia (solo señalados)

- 1. **Tooltip vs motor (53/61 acciones):** ActionCard.tsx:113-131 muestra "Apoyan/Se oponen" desde `affectedGroups`, pero el engine solo aplica `explicitGroupEffects` (8 acciones). El jugador ve apoyo/oposición prometida que **nunca ocurre**; además las 8 acciones explícitas a veces contradicen su propio `affectedGroups` (e.g. `subsidios\_industriales` lista supports empresarios+sindicatos — coincide — pero sus opposes ongs –6/fin –9 no figuran como tales).
- 2. **Matcher textual 100% muerto:** solo 4 coincidencias reales y todas en acciones con explicitGroupEffects (que tienen prioridad). Para el resto, cero efecto grupal.
- 3. **`satisfiesDemand` (44 acciones) es campo muerto:** nadie en el engine lo lee; el match de demanda es por ID de acción (`demandActionIds`/`completedActions`), no por texto.
- 4. **Rama legacy de agendas (AGENDA_TEMPLATES) inalcanzable:** los 7 grupos con templates tienen demandActionIds → siempre se usa el sistema nuevo.
- 5. **Doble vía de satisfacción asimétrica:** cumplir la demanda ejecutando la acción da +10 SIN castigo a antagonistas; el botón da +5 CON castigo. Incentiva siempre la vía acción.
- 6. **Pasiva empresario "+1 préstamo (máx 4)" no cableada:** `getMaxLoans` existe pero turnProcessor.ts:366 usa `Math.min(3,...)` (TODO reconocido en archetypeEngine.ts:83-86).
- 7. **Comentario de código auto-contradictorio:** gameEngine.ts:242 dice "getInitialState() aplica las pasivas con 'político'" pero las líneas 152-156 explican que ya NO se aplican ahí (comentario stale).
- 8. **Reforma impositiva bloqueada por diseño:** minLegislativeSupport:45 declarado "imposible de alcanzar" en el propio registry (líneas 137-139) y mantenido deliberadamente.
- 9. **actionCategories.ts es zombie** (reconocido en su propio encabezado y en actionRegistry.ts:70-73): solo se usa para `getAllActionIds` en gameEngine.ts:160-162; contiene cooldowns/efectos que no se ejecutan.
- 10. **Docs desactualizados:** `session-summary.md:29` dice "satisfacer demanda da +5... +1 popularidad" (correcto) pero `mecanicas-del-juego.md:121` describe impacto cruzado genérico; `difficulty-analysis.md:291` muestra pseudocódigo (`state.groupRelations[antagonistId] -= change \* ratio`) sin el `Math.round` real ni el clamp de una sola pasada.

3. Sistema ideológico y asesores

Radiografía de solo lectura del juego "GobernArg" (TypeScript, MVP presidente-only). Repo auditado: `project-bolt-sb1-dzgqso GobernArg 21-11/project/src`, rama `redisenovisual`. Rutas citadas relativas a `project/src`. Separación tajante entre "tiene efecto mecánico real" y "es solo texto/UI".

PARTE 1 — SISTEMA IDEOLÓGICO

1.1 Ejes existentes

NO existe un perfil ideológico de "izquierda/derecha" ni estatismo/mercado. Existen **3 ejes bipolares independientes**, rango **-100 a +100**, con clamp en `engine/axisEngine.ts:21-23` (`clampAxis`):

Eje	Campo en `GameState`	Polo -100	Polo +100	Definido en
Radical ↔ Conciliador	radicalConciliadorAxis	Radical	Conciliador	types/game.ts:509, engine/axisEngine.ts
Populista ↔ Técnico	populistaTecnicoAxis	Populista	Técnico	types/game.ts:510
Cerrado ↔ Convocante	cerradoConvocanteAxis	Cerrado	Convocante	types/game.ts:511

Inicialización: los 3 en **0** en `engine/gameEngine.ts:141-143` (`getInitialState`).

1.2 Clasificación ideológica de las acciones

No hay tags ideológicos por acción (ningún campo `ideology`/`tags` en `GameAction`, `types/game.ts:50-90`). La clasificación es **por categoría de acción** mediante la tabla dura `CATEGORY\_AXIS\_SHIFTS` en `engine/axisEngine.ts:9-19`:

Categoría	Radical/Concil.	Populista/Téc.	Cerrado/Convoc.	# acciones (`data/actionRegistry.ts`, `grep -c`)
economia	0	+2	-1	12
social	+1	-2	0	10
seguridad	-3	0	-2	7
diplomacia	+2	0	+3	7
cultura	+2	-1	+2	8
infraestructura	0	+1	0	14
educacion	+1	0	+1	1

Categoría	Radical/Concil.	Populista/Téc.	Cerrado/Convoc.	# acciones (`data/actionRegistry.ts`, `grep -c`)
turismo	0	-1	+1	1
tecnología	0	+1	0	1

Total: **61 acciones** en `data/actionRegistry.ts` (fuente única de verdad; `data/actionCategories.ts:1-43` se deriva del registry agrupando por categoría).

1.3 Cálculo del perfil

Fórmula (todo suma fija, sin pesos variables ni normalización por magnitud de acción):

1. **Al ejecutar cada acción** (dentro del procesamiento de fin de turno): `applyAxisShift(action, state)` — llamado desde `engine/turnProcessor.ts:372`, definido en `engine/axisEngine.ts:25-33`. Suma el shift de la categoría de la acción a cada uno de los 3 ejes, con clamp ±100. Cada acción pesa igual sin importar su `popularityChange`/`budgetChange`.

2. **Al inicio de cada turno, por pasiva de arquetipo**: `applyArchetypePassives(state)` — `engine/turnProcessor.ts:312` → `engine/archetypeEngine.ts:20-23,55-70`, con shifts por turno declarados en `data/specialAbilities.ts:170-194`:

- político: conciliador **+2**, convocante **+1** /turno
- empresario: técnico **+2**, cerrado **-1** /turno
- sindicalista: radical **-2**, populista **-2** /turno
- comunicador: convocante **+2** /turno

Nota: estos shifts se **re-aplican cada turno** (los acumuladores `\_archetype\*` se resetean al inicio de `applyArchetypePassives`, `archetypeEngine.ts:12-18`, pero los axis-shift no se compensan), por lo que el arquetipo empuja los ejes de forma monótona y solo las acciones ejecutadas los corrigen. No existe decaimiento natural de los ejes.

1.4 PREGUNTA CENTRAL — Consumidores del perfil

Sí hay consecuencias mecánicas, pero únicamente en los extremos (±80). `getAxisModifiers()` — `engine/axisEngine.ts:43-70` — devuelve `{}` si ningún eje llega a ±80.

Consumidor	Archivo:función:línea	Efecto real
Multiplicador de efectividad y costo de acciones	utils/actionEffects.ts:64-72 en calculateActionEffects	Radical≤-80: seguridad ×1.10, diplomacia ×0.85. Conciliador≥80: costo cultura -1%, diplomacia -1%, seguridad +1%. Populista≤-80: social ×1.20, costo social -1%. Técnico≥80: economía ×1.20, social ×0.90
Estabilidad + relaciones grupales por turno	engine/turnProcessor.ts:456-480	Cerrado≤-80: stability +5, todas las groupRelations -10/turno. Convocante≥80: groupRelations +10, stability -5/turno
Notificaciones de aviso	engine/turnProcessor.ts:83-136	Avisos al cruzar ±80 en cada eje (solo texto/notificación)
UI legado	components/LegacyScreen.tsx:196-227	3 AxisBar (Radical/Conciliador, Populista/Técnico, Cerrado/Convocante) en la pantalla final de legado

NO consumen los ejes (verificado por grep exhaustivo sobre `src/`): `engine/electionEngine.ts`, `utils/electionSystem.ts`, `engine/eventResolver.ts` (`EventConditions` en `types/game.ts:320-336` no tiene campos de ejes), `engine/groupAgendaEngine.ts`, objetivos/condiciones de victoria, `engine/actionEngine.ts`, prerequisites de acciones (`GameAction.prerequisites`, `types/game.ts:84-89`, solo acciones/apoyo legislativo/legitimidad/apoyo grupal). **No hay acciones bloqueadas ni eventos condicionados por ideología.**

1.5 ¿Reaccionan los grupos a la ideología?

Solo a través del eje Cerrado/Convocante en ±80: `turnProcessor.ts:463-470` aplica `groupRelationsModifier` de **±10 a todos los grupos por igual** (sin afinidad ideológica individual). No existe matching de ideología del jugador vs. ideología de grupo: los `groupBonuses` de los asesores tampoco se aplican (ver Parte 2). Los grupos reaccionan por intereses textuales (`utils/actionEffects.ts:144-170`, matching de palabras de `subgroup.interests` contra la descripción de la acción), no por espectro político.

PARTE 2 — ASESORES

2.1 Listado completo (`data/advisors.ts:3-171`)

ID	Nombre	Especialidad	bonusActions	Costo (\$M)	popularityEffect	unlockRequirement	level	influence	effectiveness
advisor1	Dr. Carlos Méndez	Economista	2	300	-5	null	3	8	85
advisor2	Lic. María González	Comunicación Social	1	250	10	null	2	9	80
advisor3	Ing. Roberto Silva	Infraestructura	2	400	5	null	4	7	90

ID	Nombre	Especialidad	bonusActions	Costo (\$M)	popularityEffect	unlockRequirement	level	influence	effectiveness
advisor4	Dra. Ana Martínez	Políticas Sociales	1	200	15	null	3	8	85
advisor5	Dr. Jorge Ramírez	Relaciones Internacionales	2	500	8	popularity ≥ 60	5	9	95
advisor6	Lic. Patricia Sánchez	Seguridad Pública	1	350	12	null	3	7	85
advisor7	Dr. Miguel Ángel Torres	Educación	2	300	10	popularity ≥ 55	4	8	90

No existe salario/upkeep: el `cost` es un pago único al contratar (`engine/gameEngine.ts:372-389`). No hay gasto por turno por mantener asesores.

2.2 Tabla campo × consumidor × efecto × estado

Tipo real: `types/game.ts:101-120` (`Advisor`) y `:122-125` (`AdvisorWithStatus`).

Campo	Consumidor	Efecto exacto	Estado
id	engine/eventResolver.ts:317-322 (requiredAdvisors en checkEventConditions)	Condición de eventos: el asesor debe estar contratado y isActive. Ningún evento en data/events/ declara requiredAdvisors (grep sin resultados fuera de tests) → consumidor vivo pero sin datos	FUNCIONA (sin uso real)
bonusActions	utils/actionCalculator.ts:23-28 en calculateAvailableActions (filtra advisor.isActive)	Suma directa a las acciones del turno: baseActions += Σ bonusActions de asesores activos. Recalc inmediato al contratar/despedir vía recalcState	FUNCIONA
specialty	utils/actionEffects.ts:114-142 (ADVISOR_SPECIALTY_CATEGORY_MAP + calculateAdvisorMultiplier)	Si la categoría de la acción está en el mapa de la especialidad (key normalizada toLowerCase().trim()): popularityChange ×1.2 por cada asesor activo que mapee (multiplier *= 1.2, línea 137, acumulativo). Solo afecta popularidad de la acción, no presupuesto ni efectos grupales	FUNCIONA (con 2 especialidades desfasadas, ver 2.4)
cost	engine/gameEngine.ts:372-373 (hireAdvisors); UI components/AdvisorSelectionModal.tsx:70-71	Pago único al contratar (budget - totalCost); rechazo silencioso si budget < totalCost	FUNCIONA
popularityEffect	Ninguno en engine. Solo getPopularityIndicator en AdvisorSelectionModal.tsx:14-26,97 y AdvisorPanel.tsx:15,78 (iconos TrendingUp/Down por signo)	—	DISPLAY-ONLY (nunca toca state.popularity)
groupBonuses	Ninguno en engine. Solo render AdvisorSelectionModal.tsx:165-169 y AdvisorPanel.tsx:133 ("X% relación con grupo")	—	DISPLAY-ONLY
policyModifiers	Ninguno en engine. Solo render AdvisorSelectionModal.tsx:160-164 y AdvisorPanel.tsx:128 ("X% efectividad en categoría")	—	DISPLAY-ONLY
specialAbilities	Ninguno. ARCHETYPE_ABILITIES/useSpecialAbility (engine/gameEngine.ts:424+) son habilidades del arquetipo, no del asesor; los strings de asesor no se resuelven en ningún lado	—	SIN CONSUMIDOR (ni siquiera se muestran en UI)
influence	Solo UI (AdvisorSelectionModal.tsx:134, AdvisorDismissModal.tsx:60)	—	DISPLAY-ONLY
level	Solo UI (AdvisorSelectionModal.tsx:125, AdvisorPanel.tsx:99)	—	DISPLAY-ONLY
traits	Solo data; ningún render ni engine	—	SIN CONSUMIDOR (ni siquiera se muestran)
effectiveness	Solo data	—	SIN CONSUMIDOR
unlockRequirement	Solo UI: AdvisorSelectionModal.tsx:73-79 (isAdvisorAvailable) filtra quién aparece para contratar	El engine no lo verifica; como el modal es la única vía de contratación, efectivamente funciona	FUNCIONA (vía UI)
description	UI	—	DISPLAY-ONLY

2.3 Reglas de contratación (`engine/gameEngine.ts:353-404`)

Regla	Implementación
Máximo de slots	2 asesores simultáneos (MAX_ADVISORS = 2, gameEngine.ts:367). El UI calcula slots libres (components/AdvisorPanel.tsx:43, maxAdvisorsToHire = 2 - advisors.length) y el engine corta la lista por slots (candidates.slice(0, slots), gameEngine.ts:368-369) en lugar de rechazar toda la operación
Anti-duplicados	Filtra ids ya contratados (gameEngine.ts:361-363); el comentario registra que antes el UI permitía re-seleccionarlos y el engine los duplicaba, duplicando bonos
Una operación por turno	advisorActionUsed gatea contratar (gameEngine.ts:357) y despedir (:396); se resetea al procesar fin de turno (engine/turnProcessor.ts:613) y al resolver elección (engine/electionEngine.ts:113)
Efecto inmediato vs próximo turno	Inmediato: ambas funciones terminan en recalcState (gameEngine.ts:384, :399), que recalcula baseActions/actions al instante (engine/engineShared.ts:110-122). Comentario explícito "FIX (Punto 12)" en :381-383 y :397-398
Despido	dismissAdvisor (gameEngine.ts:392-404): elimina por id, sin reembolso, recalc inmediato, consume la acción de asesor del turno

2.4 Especialidades × mapeo a acciones

Mapa real en `utils/actionEffects.ts:115-127` (`ADVISOR\_SPECIALTY\_CATEGORY\_MAP`), con normalización `advisor.specialty.toLowerCase().trim()` (`:134`):

Especialidad en data (`data/advisors.ts`)	Key normalizada	Categorías mapeadas	Bonus ×1.2 alcanzable
Economista	economista	economia	■
Comunicación Social	comunicación social	cultura, social, diplomacia	■
Infraestructura	infraestructura	infraestructura	■
Políticas Sociales	políticas sociales	NINGUNA (no está en el mapa)	■
Relaciones Internacionales	relaciones internacionales	diplomacia	■
Seguridad Pública	seguridad pública	NINGUNA (el mapa tiene seguridad, sin "pública")	■
Educación	educación	educacion	■ (pero solo 1 acción de educación existe en el registry)

Confirmado: **2 de 7 especialidades ("Políticas Sociales" y "Seguridad Pública") jamás otorgan el ×1.2** por desfasaje de nombre. El bonus es acumulativo por asesor que mapee (`multiplier \*= 1.2`, `actionEffects.ts:137`).

2.5 Efectos no declarados

- Contratar/despedir recalcula las acciones disponibles **al instante** (`recalcState`, ver 2.3). `recalcState` también recalcula `popularity` vía `calculatePopularidad` y `votingIntention` vía `calculateElectionVotingIntention` (`engine/engineShared.ts:110-121`).
- `turnsInactive` / `isActive: false`: **ningún código del engine** escribe estos campos con valor distinto de `0`/`true` (grep `isActive: false|turnsInactive: [1-9]` solo arroja tests y docs). Los asesores **nunca se inactivan**: es un estado muerto que solo la UI sabe mostrar (`components/AdvisorPanel.tsx:122`, `components/AdvisorDismissModal.tsx:53`). El filtro `advisor.isActive` de `actionCalculator.ts:25` y `actionEffects.ts:133` evalúa siempre true en la práctica.

Hallazgos de consistencia (solo señalados, sin proponer cambios)

1. **La UI promete bonos que no existen.** `AdvisorSelectionModal.tsx:157-171` muestra "+30% efectividad en economía" (policyModifiers) y "+15% relación con empresarios" (groupBonuses), y `popularityEffect` se grafica con flechas de tendencia. Ninguno de los tres campos es leído por ningún engine. El jugador paga \$200-500M por estadísticas decorativas; el único retorno mecánico real de un asesor es `bonusActions` (acciones extra) y el ×1.2 de especialidad sobre la popularidad de la acción.
2. **Desfasaje de nombres de especialidad:** `data/advisors.ts` usa "Seguridad Pública" y "Políticas Sociales"; el mapa de `actionEffects.ts:115-127` no las reconoce → esos dos asesores nunca aplican su único bonus mecánico posible (el ×1.2). El jugador que contrata a Patricia Sánchez por su especialidad de seguridad no recibe nunca el bonus en las 7 acciones de seguridad.
3. **Magnitud casi nula de los costos por eje:** `axisEngine.ts:50-51` define `actionCostModifier` de ±1, y `actionEffects.ts:67-68` lo convierte en `1 + modifier/100` = **±1% de costo**, mientras los `effectivenessMultiplier` sí son ×1.10-1.20. Las notificaciones alarmistas de `turnProcessor.ts:83-136` ("Gobierno radicalizado", "Aislamiento político") sugieren consecuencias fuertes de los extremos ideológicos; en costo de acciones el efecto es de 1%.
4. **Doc de diseño describe mecánicas ausentes:** `docs/details/asesores\_version\_detallada.txt` documenta un `calculateAdvisorEffects` con `popularityModifier` aplicado a la popularidad, desactivación de asesores por `turnsInactive` y reactivación decrementando turnos — nada de eso existe en el engine actual.
5. **Plan vs. realidad de UI ideológica:** `docs/implementation-plan.md:78` prevé una sección "Perfil Ideológico" con 3 AxisBar en `IndicatorsPanel`; `components/IndicatorsPanel.tsx` real no muestra ejes (grep sin coincidencias de Axis/ideología) — el perfil solo es visible en `LegacyScreen` (pantalla final de legado), siendo ciego durante toda la partida pese a tener efectos mecánicos al cruzar ±80.
6. **`requiredAdvisors` es infraestructura vacía:** existe en el tipo (`types/game.ts:328`, también `systems/events/types.ts:33`) y en `eventResolver.ts:317-322`, pero ningún evento en `data/events/` lo declara (solo un test sintético en `\_\_tests\_\_/eventResolver.test.ts`).
7. **Deriva ideológica monótona por arquetipo:** las pasivas de arquetipo aplican shift de eje **cada turno** sin compensación (`archetypeEngine.ts:55-70`, corrido desde `turnProcessor.ts:312`). Un jugador que ejecuta pocas acciones ve cómo su arquetipo solo empuja un eje hacia el extremo de forma acumulativa (p. ej. sindicalista: -2/-2 por turno llega a -80 en ~40 turnos), y desde ahí los modificadores de extremo operan solos.
8. **UI de asesores inactivos muerta:** `AdvisorPanel.tsx:122` ("Estado: Activo / Inactivo por N turnos") y `AdvisorDismissModal.tsx:53` renderizan estados de inactividad que jamás ocurren en el engine (punto 2.5), sugiriendo al jugador una mecánica de rotación/sanción inexistente.

9. El **×1.2 de asesor no refleja lo que la tarjeta sugiere**: el bonus de especialidad solo multiplica `popularityChange` (`actionEffects.ts:72`), no `budgetChange` ni `groupEffects`, mientras la tarjeta del asesor en UI insinúa "efectividad" general por categoría (policyModifiers mostrados, que además no se aplican). El tooltip de la acción y el resultado real coinciden entre sí, pero no con la promesa de la tarjeta del asesor.

4. Sistemas muertos, redundantes y contradicciones UI vs código

Radiografía de solo lectura del MVP presidente-only (branch `rediseño-visual`).

Inventario EXACTO de (1) variables y sistemas muertos, redundantes, decorativos o desconectados y (2) contradicciones entre UI/textos y comportamiento real. Sin propuestas de cambio: solo inventario y señalamiento, con referencias `archivo:función:línea`.

Convención: líneas aproximadas al estado actual del branch. Verificación por grep/lectura directa sobre `project-bolt-sb1-dzgqso GobernArg 21-11/project/src`.

PARTE 1 — SISTEMAS MUERTOS / DECORATIVOS / REDUNDANTES / DESCONECTADOS

1A. Verificados de la lista inicial

#	Sistema/variable	Dónde se define	Qué debería hacer	Estado real	Evidencia	Quién lo muestra en UI
1	getMaxLoans / _archetypeExtraLoans	engine/archetypeEngine.ts:getMaxLoans:87-89 (acumulador en :36-38)	Límite de préstamos 3 + extra por pasiva empresario	MUERTO/DESCONECTADO: nunca se llama; el límite real es Math.min(3, ...)	engine/turnProcessor.ts:366 (debtCount = Math.min(3, ...)); TODO propio en archetypeEngine.ts:83-85	Pasiva "Red de contactos: 1 préstamo extra (máx 4)" (data/specialAbilities.ts:183) mostrada en tooltip de components/CharacterCreation.tsx:101-114 — falsa
2	difficulty.baseActionsModifer	engine/difficultyEngine.ts:11,20,30,38,46,54	Modificar puntos de acción base por dificultad	MUERTO: sin consumidores (lo admite el comentario del propio archivo :11)	grep: solo definición	—
3	difficulty.ironman	engine/difficultyEngine.ts:12,19,29,39,47,53	Modo ironman (sin guardado/permadeath)	MUERTO: sin consumidores	grep: solo definición	—
4	Dificultad no-normal completa	engine/difficultyEngine.ts:24-57; engine/gameEngine.ts:createNewGame:187 (default 'normal')	easy/hard/legend modificarían decay, ingreso, crisis, préstamos	DESCONECTADO/INALCANZABLE: la UI nunca pasa dificultad → siempre 'normal' (App.tsx:73 llama createNewGame(position, archetype, governorName, false, avatar) sin el 6º parámetro). Sin selector de dificultad en ningún componente	grep difficulty en components/: solo tests y tipos	—
5	Pasiva comunicador incomeBonus: 0	data/specialAbilities.ts:193 ("Agenda setting")	Dato de la pasiva debería producir el efecto	REDUNDANTE/DECORATIVO: el ×1.1 está hardcodeado en getArchetypeMultiplier (utils/actionEffects.ts:107-108); el campo vale 0 y no produce nada	ambos archivos citados	Tooltip de pasivas en CharacterCreation.tsx:104
6	eventWeights.ts (79 líneas)	data/events/eventWeights.ts:2-79; re-export en data/events/index.ts:5,23	Ponderar selección de eventos por categoría /severidad/arquetipo/cargo/estado/turno	MUERTO: ningún consumidor; resolveRandomEvents (engine/eventResolver.ts:218-296) no usa pesos ni weight	grep eventWeights: solo definición y re-export	—
7	satisfiesDemand (~60 entradas)	data/actionRegistry.ts:53 (tipo) y :105...1137 (datos)	Marcar qué demandas satisface cada acción	MUERTO: cero consumidores en src (el cumplimiento real es por completedActions.includes(agenda.demand), engine/groupAgendaEngine.ts:155)	grep: solo definiciones/datos	—
8	affectedGroups.supports/opposes	data/actionRegistry.ts:48-51 (en todas las acciones)	Declarar grupos beneficiados/afectados	DECORATIVO: solo tooltip; los efectos reales vienen de explicitGroupEffects o del matcher textual (utils/actionEffects.ts:144-170) + matriz unidireccional GROUP_ANTAGONISTS (utils/crossGroupEffects.ts)	components/ActionCard.tsx:113-133	Tooltip "Grupos afectados" en ActionCard
9	Matcher textual de grupos	utils/actionEffects.ts:calculateGroupEffects:151-169	Matching intereses↔descripción como fuente de efectos	VIVO pero frágil (fallback solo para acciones sin explicitGroupEffects; includes() sobre descripción en minúscula)	actionEffects.ts:156-158	(indirecto: tooltip no refleja este matching)

#	Sistema/variable	Dónde se define	Qué debería hacer	Estado real	Evidencia	Quién lo muestra en UI
1 0	Impeachment	utils/victoryConditions.ts:checkAllDefeatCondi tions:62-66	Derrota por pop<10 + estab<20 ×2 turnos	MUERTO (sombreado): pop<10 ⇒ pop<30 (umbral presidente) ⇒ low_popularity dispara en los mismos 2 turnos y se evalúa antes (:52-54); la rama impeachment jamás se alcanza	victoryConditions.ts:52-66; tracking en engine/turnProcessor.ts:181-185	data/defeatReasons.ts:26-34 (texto y consejo inalcanzables); icono en components/GameOverModal.tsx:16
1 1	Golpe institucional	utils/victoryConditions.ts:69-73	Derrota por estab<10 + legislativeSupport<25 ×3 turnos	MUERTO (imposible): legislativeSupport se clampa a [25,75] (engine/eventResolver.ts:75), y cuando es null usa ?? 100	eventResolver.ts:74-75; turnProcessor.ts:186	data/defeatReasons.ts:36-42; GameOverModal.tsx:17
1 2	legislativeSupport	types/game.ts:486; set en engine/eventResolver.ts:108	Bancada propia que modifica costos de reforma	VIVO pero limitado (corrección al supuesto "siempre null"): se calcula en legislativas de año 2T4 de cada mandato y se resetea a null tras cada elección (engine/electionEngine.ts:99). Año 1 de cada mandato: null ⇒ penalty 0	Consumidores: actionEngine.ts:16-30,50, engineShared.ts:130, turnProcessor.ts:186	Banner en components/ControlPanel.tsx:50-87
1 3	POSITION_ACTION_EX CLUSIONS	engine/engineShared.ts:46-50	Excluir acciones por cargo	REDUNDANTE: las tres listas vacías; la disponibilidad real la da availableForPositions (filtrado en engine/actionEngine.ts:38)	engineShared.ts:44-45 (comentario propio lo reconoce)	—
1 4	DEFEAT_POP_THRES HOLD duplicado	engine/engineShared.ts:52-56 y utils/victoryConditions.ts:4-8	Umbral de derrota por popularidad	REDUNDANTE: dos tablas 20/25/30 idénticas, doble fuente de verdad	ambos archivos	—
1 5	getInitialGameState actions:5 / baseActions:5	engine/gameEngine.ts:90-91	Acciones iniciales	MUERTO/ENGAÑOSO: pisado inmediatamente en createNewGame:239-240 por calculateAvailableActions; la base real de presidente es 1 (utils/actionCalculator.ts:5-9) + bonuses	gameEngine.ts:239-240; actionCalculator.ts:5-35	components/GameHeader.tsx:128 muestra el valor vivo
1 6	Código de carrera "MODO CAMPAÑA"	data/careerRules.ts (todo el archivo, STARTING_POSITION:10), utils/electionSystem.ts:45-52 (promote-*), engine/electionEngine.ts:72-93, utils/victoryConditions.ts:154-226 (objetivos intendente/gobernador), utils/ascensionPenalty.ts, engine/turnProcessor.ts:444-448 (decay por cargo), engine/engineShared.ts:18-34 (income/maintenance/budget por cargo), tests __tests__ /careerRules.test.ts	Carrera intendente→gobernador→presidente	VIVO PERO INACCESIBLE (solo position=== 'presidente' alcanzable; todo lo demás queda detrás del flag STARTING_POSITION)	comentarios "MODO CAMPAÑA (RESERVADO POST-MVP)" en careerRules.ts:1-4, electionSystem.ts:41-43, electionEngine.ts:64-66, victoryConditions.ts:150-153	ReelectionChoiceModal solo ofrece reelección (promote-* nunca aparecen); LegacyScreen/careerLog.ts generan narrativa de saltos de cargo que nunca ocurren

1B. Variables de GameState: fuentes y consumidores reales

#	Variable	Fuentes (qué la modifica)	Consumidores reales	Estado	UI
1 7	socialConflict / Conflicto social	No existe como variable. Derivado: deriveConflictoSocial (moods conflictivos×8 + demandas vencidas×5 + lowPop×4 + negBudget×3)	Ninguno (ni eventos ni derrotas lo leen)	DECORATIVO	components/IndicatorsPanel.tsx:31-52,274-283 (tarjeta "Conflicto Social" con target 40 que no dispara nada). Los eventos de data/events/social.ts tocan stability/popularity, no un índice de conflicto
1 8	legitimacy	calculateLegitimacyChange (engine/legitimacyEngine.ts:4-24, aplicado en turnProcessor.ts:370-371): +3 cultura/diplomacia, −8 decretos, −min(5,.) impopular; multiEffects.legitimacyChange de acciones (data/actionRegistry.ts:78); habilidades (gameEngine.ts:449-451,464-466)	Prerequisitos minLegitimacy (actionEngine.ts:52-54); costos de habilidades; rating del legado (LegacyScreen.tsx:37)	VIVO pero con sub-sistema muerto: checkLegitimacyCostMultiplier (legitimacyEngine.ts:27-29) sin consumidores	IndicatorsPanel.tsx:263-272; LegacyScreen; SpecialAbilitiesPanel

#	Variable	Fuentes (qué la modifica)	Consumidores reales	Estado	UI
1 9	stability	multiEffects de acciones; efectos de eventos (eventResolver.ts:398-399); outcome legislativo (eventResolver.ts:112-125); estrategias midterm (turnProcessor.ts:422); ejes extremos (turnProcessor.ts:459-462); habilidades; pendingEffects.stabilityChange	Condiciones de eventos min/maxStability (eventResolver.ts:304-305); checks impeachment/coup; calculateLegislativeResults (eventResolver.ts:55); intención de voto (peso 0.05, electionSystem.ts:121); legado	VIVO	GameHeader.tsx:176; IndicatorsPanel.tsx:252-261; EventModal
2 0	votingIntention (intención de voto)	Fuente única: calculateElectionVotingIntention recalculada en recalcState cada turno (engine/engineShared.ts:120); pisada con el resultado electoral (electionEngine.ts:50)	UI (IndicatorsPanel, RightSidebar); proyección de reelección (electionSystem.ts:62-81); umbral de victoria 45% (electionSystem.ts:21,88)	VIVO, pero multiEffects.votingIntentionChange (data/actionRegistry.ts:79,851) es MUERTO: turnProcessor.ts:337 lo aplica y el paso 8 (recalcState, turnProcessor.ts:565) lo sobrescribe calculándolo de cero	IndicatorsPanel "Intención de Voto" target 45; RightSidebar "Riesgo derrota"
2 1	Coaliciones	Evento coalition_opportunity con choices accept/reject (data/events/political.ts:5-52) — VIVO; estrategia abrirse	Estrategia: +5 estab/+1 pop por turno, ×1.15, −2 aliados/turno (turnProcessor.ts:438-440)	Evento vivo; estrategia no crea ninguna coalición pese al texto "Armás coaliciones amplias" (data/midtermStrategies.ts:30)	MidtermStrategyModal; costo −2 aliados invisible (aliados excluido del acordeón y de Situación Electoral: lib/groups-mapping.ts:95, RightSidebar.tsx:42-43,74,78)

1C. Campos de datos sin consumidor (verificación campo por campo)

#	Campo	Definición	Estado	Evidencia
2 2	GameAction.unlockedActions / GameState.unlockedActions	types/game.ts:60,495; init gameEngine.ts:118 (getAllActionIds() = todas)	MUERTO: nadie modifica la lista; el filtro actionEngine.ts:79 pasa siempre. calculateActionEffects lo devuelve (actionEffects.ts:90) sin consumidor	grep unlockedActions
2 3	ActionDefinition.triggersEvent	data/actionRegistry.ts:52	MUERTO: sin consumidores	grep triggersEvent
2 4	EventChoice.consequences	types/game.ts:359-362; systems/events/types.ts:55-58	MUERTO: sin consumidores (applyEventChoice solo lee effects)	engine/eventResolver.ts:343-382
2 5	Subgroup.supportMultiplier, resourceDemand, satisfactionLevel, lastInteractionEffect	types/game.ts:140-143; datos en data/interestGroups.ts	MUERTO: decorativos; el engine usa solo baseSupport, influence, interests, demandActionIds	grep campo por campo
2 6	Subgroup.demands (texto)	types/game.ts:137; interestGroups.ts	DECORATIVO: las agendas usan AGENDA_TEMPLATES propio o demandActionIds (groupAgendaEngine.ts:5-13,34), nunca sg.demands	groupAgendaEngine.ts
2 7	Advisor.popularityEffect, .level, .influence, .groupBonuses, .policyModifiers, .specialAbilities (ids), .traits, .effectiveness	types/game.ts:109-119; data/advisors.ts	DECORATIVOS: solo se muestran (o ni se muestran: traits, effectiveness). Los únicos con efecto real: bonusActions (actionCalculator.ts:24-26), specialty vía ADVISOR_SPECIALTY_CATEGORY_MAP (actionEffects.ts:115-142), cost, unlockRequirement	components/AdvisorPanel.tsx:78-138, AdvisorSelectionModal.tsx:74-76,97,125,134,160-166
2 8	Specialty sin match: 'Políticas Sociales' (advisor4), 'Seguridad Pública' (advisor6)	data/advisors.ts:76,126 vs actionEffects.ts:115-127	DESCONECTADO: esas specialties no están en el mapa → el ×1.2 de asesor nunca aplica para ellas	actionEffects.ts:134-138
2 9	AdvisorWithStatus.isActive:false / turnsInactive	types/game.ts:123-124	MUERTO: hireAdvisors siempre crea isActive:true, turnsInactive:0 (gameEngine.ts:375-379); nada los cambia. Rama "Inactivo por X turnos" inalcanzable	AdvisorPanel.tsx:121-123
3 0	GameState.isAdminMode	types/game.ts:491; gameEngine.ts:114,185,217; App.tsx:62-73	MUERTO: siempre false (WelcomeScreen llama onStart(false)), jamás leído por engine/UI	grep isAdminMode (solo creación y docs viejos)
3 1	GameState.popularidadGrupos / popularidadPolitica	set en recalcState (engineShared.ts:113-114)	DECORATIVOS: ningún componente los muestra ni los consume	grep
3 2	GameState.termsByPosition	gameEngine.ts:81; usado solo en ramas promote-*	INACCESIBLE en MVP (ver #16)	electionSystem.ts:72-76
3 3	TurnLogEntry.decisions	types/game.ts:262; siempre [] (turnProcessor.ts:603)	MUERTO: "se completan externamente" pero nadie lo hace	turnProcessor.ts:603
3 4	PendingEffect.target/value/conditions/type/source (los no usados en traducción)	types/game.ts:293-311	target/value se traducen en applyEventChoice (eventResolver.ts:360-377); conditions, type (solo display vía inferEffectType), source (display) sin efecto mecánico	parcial
3 5	systems/effects/types.ts (módulo completo: Effect, EffectState, HistoricalEffect...)	systems/effects/types.ts:1-33	MÓDULO ZOMBIE: cero imports en todo src	grep systems/effects

#	Campo	Definición	Estado	Evidencia
3 6	narrativeEngine.generateActionResult / generateInteractionResult / generateElectionResult	engine/narrativeEngine.ts:102,132,152	MUERTOS: solo generateTurnIntro se usa (turnProcessor.ts:285)	grep
3 7	getAntagonistsForGroup	data/groupAntagonists.ts:45-47	MUERTO: crossGroupEffects lee GROUP_ANTAGONISTS directo	utils/crossGroupEffects.ts:16
3 8	processElectionResults (wrapper) y checkDefeatConditions (boolean)	utils/electionSystem.ts:103-106; utils/victoryConditions.ts:21-39	MUERTOS en producción: solo los usan tests; el juego usa processElectionResultsForOption y checkAllDefeatConditions	grep
3 9	Event.weight (varios eventos) y GameEvent.weight	datos (economic.ts:50, pendingEvents.ts:165...); types/game.ts:376	MUERTO: nadie pondera por weight	grep weight
4 0	Notification campos expiresAt, groupId, requiresAcknowledgment, metadata, actions	types/game.ts:415-421	MUERTOS: addNotification nunca los setea; NotificationCenter solo usa id/title/message/importance/read/dismissed/year/turn	engineShared.ts:90-108; components/NotificationCenter.tsx
4 1	CalendarEvent.effect	types/game.ts:439	MUERTO: ningún evento de data/calendar.ts define effect; los efectos reales están hardcodedados por id (eventResolver.ts:137,149)	data/calendar.ts
4 2	Cooldown de interacciones (interactionHistory)	gameEngine.ts:340-343 (set turnsLeft:2), decremento turnProcessor.ts:536-542	SEMIMUERTO: la UI bloquea los botones (RightSidebar.tsx:411-412,473-474) pero applyInteraction (gameEngine.ts:248-351) no verifica el lock — el engine lo ignora por completo	ambos

PARTE 2 — CONTRADICCIONES UI/TEXTOS vs COMPORTAMIENTO REAL

#	Qué dice la UI	Qué hace el código	Dónde (UI vs engine)
C 1	Barra "Probabilidad de éxito 70%/80%..." en elecciones de eventos	choice.probability nunca se lee: applyEventChoice aplica effects.immediate siempre. Éxito real = 100%	components/EventModal.tsx:161-162,208-226 vs engine/eventResolver.ts:applyEventChoice:343-382 (no referencia probability)
C 2	ActionCard muestra efectos crudos: "+20% popular.", "-\$300M", y tooltip "Costo real" con valores crudos	Efecto real = popularityChange × 0.40 × (multiplicador arquetipo × asesor × estrategia × reunión × rendimiento decreciente × ejes); presupuesto real con diminishingFactor × costMultiplier (y mantenimiento diferido -15% oculto). La UI sobreestima sistemáticamente	lib/format.ts:formatImmediateEffect:25-44, components/ActionCard.tsx:94,149-157 vs utils/actionEffects.ts:43,71-84,206-216
C 3	Tooltip legitimidad: "Si baja a 0, las acciones cuestan el doble"	checkLegitimacyCostMultiplier existe pero nadie lo llama; a legitimidad 0 nada cambia	components/IndicatorsPanel.tsx:270-272 vs engine/legitimacyEngine.ts:27-29 (sin consumidores)
C 4	Pasiva empresario: "Eficiencia económica: Acciones de economía generan +20% presupuesto"	incomeBonus: 0.20 multiplica el ingreso fiscal por turno (POSITION_INCOME), no el presupuesto de las acciones de economía	data/specialAbilities.ts:182 vs engine/turnProcessor.ts:406-407
C 5	Pasiva político: "Constructor de alianzas: Reuniones con aliados no cuestan dinero"	El grupo aliados no es interactuable: flattenSubgroups descarta ids sin SUBGROUP_TO_GROUP y aliados/opositores están excluidos del acordeón. Pasiva sin objeto de aplicación	data/specialAbilities.ts:178; components/RightSidebar.tsx:40-43; lib/groups-mapping.ts:95
C 6	Pasiva empresario: "Red de contactos: puede tomar 1 préstamo extra (máx 4)"	Límite real Math.min(3, ...); getMaxLoans nunca se usa ⇒ máx 3 siempre	data/specialAbilities.ts:183 vs engine/turnProcessor.ts:366
C 7	CharacterCreation: Político "+2 acciones por turno"	Base presidente = 1, bono político = +1 ⇒ 2 totales, no "+2 de bonus" (la tarjeta lo presenta como bonus sobre una base implícita de 5 del estado inicial)	components/CharacterCreation.tsx:50 vs utils/actionCalculator.ts:5-17
C 8	IndicatorsPanel: tarjeta Popularidad con "Target: 40" (presidente) y tooltip "si baja del umbral del cargo (40%) por 2 turnos... perdés"	La derrota real es popularity < 30 ×2 turnos para presidente (DEFEAT_POP_THRESHOLD). El "target" 40 y el tooltip mienten el umbral	components/IndicatorsPanel.tsx:24-28,248-249 vs engine/engineShared.ts:52-56, utils/victoryConditions.ts:4-8
C 9	Tooltip presupuesto: "Déficit sostenido (3 turnos consecutivos) puede derivar en crisis"	Derrota por déficit a los 2 turnos (NEGATIVE_BUDGET_TURNS: 2)	components/IndicatorsPanel.tsx:197 vs utils/victoryConditions.ts:10,57-59
C 10	Aviso "Inestabilidad política: los eventos negativos serán más frecuentes"	La estabilidad solo filtra eventos con condición min/maxStability por evento; no hay multiplicador global de frecuencia (el único multiplicador de probabilidad es por dificultad, siempre 'normal'). eventWeights.stateModifiers.lowStability existe pero está muerto	engine/turnProcessor.ts:54-62 vs engine/eventResolver.ts:218-296
C 11	Banner legislativo "Mayoría aplastante: las reformas cuestan menos esfuerzo político" y badge REC "costo reducido" (getRecReason)	Con support ≥45 el penalty es -1 ⇒ actionCost = max(1, 1-1) = 1, idéntico al quorum (penalty 0 ⇒ 1). La mayoría aplastante no reduce nada; solo paridad (+1⇒2) y derrota (+2⇒3) cambian el costo	components/ControlPanel.tsx:68-71, lib/format.ts:80-82 vs engine/actionEngine.ts:16-22,82-94
C 12	Banner "Congreso hostil: las reformas grandes cuestan +2 acciones o están bloqueadas"	Nada las bloquea: solo cuestan 3 acciones	components/ControlPanel.tsx:81-84 vs engine/actionEngine.ts:82-94

#	Qué dice la UI	Qué hace el código	Dónde (UI vs engine)
C 13	Modal estrategia "Negociar": muestra "Costo extra +1 acc." (actionCostModifier: +1)	actionCostModifier nunca se consume por el engine; seleccionar acciones cuesta lo de siempre	components/MidtermStrategyModal.tsx:163-172, types/game.ts:218 vs engine/actionEngine.ts:toggleActionSelection:97-115 (no usa la estrategia)
C 14	Estrategia "Abrirse": "Armás coaliciones amplias... Costo: cedés apoyo en tu base (−2 aliados por turno)"	Efecto real: +5 estab/+1 pop/turno, ×1.15, y −2 a groupRelations["aliados"] — grupo no visible en acordeón ni en Situación Electoral (costo invisible). No se crea coalición alguna	data/midtermStrategies.ts:20-31 vs engine/turnProcessor.ts:438-440; RightSidebar.tsx:74,78
C 15	Acordeón de demandas muestra el texto de la demanda	agenda.demand es un actionId crudo en el sistema nuevo ("reduccion_gasto", "plan_viviendas") — se imprime sin resolver título	components/RightSidebar.tsx:443-445 vs engine/groupAgendaEngine.ts:107 (guarda el id)
C 16	Botón "Satisfacer demanda" (+5 apoyo, +1 pop, 1 acción) vs cumplir ejecutando la acción demandada (+10 apoyo)	Dos vías de resolución con recompensas distintas y sin explicarlas; además completedActions no se limpia dentro del mandato, así que ejecutar la acción antes de que exista la demanda también la cumple	engine/gameEngine.ts:satisfyGroupDemand:480-534 (+5) vs engine/groupAgendaEngine.ts:applyGroupSatisfactionPenalty:155-163 (+10)
C 17	ActionCard badge "{n}t cd"	Muestra solo el cooldown declarado; los cooldowns por defecto (getDefaultCooldown: préstamos/emisión 8t, >\$500 6t, ≥\$200 3t) no se muestran; y las acciones en cooldown ni siquiera aparecen (ver C18)	components/ActionCard.tsx:234-239 vs utils/actionEffects.ts:257-265, engine/actionEngine.ts:40
C 18	Existe UI de "Bloqueada (BLQ)" con razones (presupuesto/popularidad/cooldown) y getBlockReason	ControlPanel solo renderiza getAvailableActionsForState — las acciones bloqueadas por cooldown/cargo/prerequisitos/presupuesto desaparecen del listado; el badge BLQ solo aparece masivamente cuando canTakeAction=false (razón genérica "No disponible"). Código de bloqueo casi muerto	components/ControlPanel.tsx:20,129-139 vs components/ActionCard.tsx:68-77, lib/format.ts:62-76
C 19	TurnSummaryModal titula "Efectos Inmediatos: Popularidad X%, Presupuesto Y"	totalPopularityChange/totalBudgetChange incluyen también el desgaste natural, los ingresos/gastos fiscales y la inflación — no son "efectos inmediatos" de las acciones; y se imprimen con % sobre valores decimales	components/TurnSummaryModal.tsx:64-104 vs engine/turnProcessor.ts:315-329,413-417,443-454,615-629
C 20	RightSidebar "Riesgo derrota" del voto: ≥50 bajo, ≥40 medio...	La victoria electoral requiere ≥45% (MIN_VOTES_TO_WIN): entre 45–49% se muestra "medio" pese a ser victoria	components/RightSidebar.tsx:63-68,101-104 vs utils/electionSystem.ts:21,88
C 21	Tooltip estabilidad: "Estabilidad baja + popularidad baja puede llevar a impeachment o golpe"	Impeachment queda sombreado por low_popularity (mismos 2 turnos, evaluado antes) y el golpe es imposible (clamp de support a ≥25). Ninguna de las dos vías ocurre jamás	components/IndicatorsPanel.tsx:259-261 vs utils/victoryConditions.ts:52-73, engine/eventResolver.ts:75
C 22	Aviso "Gobierno radicalizado / populismo extremo / aislamiento político..." para ejes ≤−80; el eje cerrado− da stabilityModifier +5 y groupRelations −10	Los textos de "Aislamiento político" (descontento en todos los sectores) vs efecto real (estabilidad +5, relaciones −10); simétricamente "Apertura total" da relaciones +10 y estabilidad −5. Los warnings describen castigo puro donde hay trade-off	engine/turnProcessor.ts:120-136 vs engine/axisEngine.ts:62-68
C 23	EventModal lista "Efectos inmediatos: popularity: +10" y badges de grupos con label crudo	Muestra effect.target sin traducir (mezcla español/inglés: "Presupuesto/Popularidad/Estabilidad" en badge vs "popularity/budget" en la lista)	components/EventModal.tsx:196-203,229-250
C 24	ElectionResultsModal muestra details.stabilityBonus como "X.X%"	Es el valor 0–100 de estabilidad, no un porcentaje de impacto	components/ElectionResultsModal.tsx:131 vs utils/electionSystem.ts:181-183
C 25	AdvisorPanel: íconos de "Popularidad" (hasta 3 flechas) y "Bonificaciones: +30% efectividad en economía / +15% relación con empresarios"	Ninguno tiene efecto: popularityEffect, policyModifiers, groupBonuses no los lee el engine. Único efecto real de asesores: bonusActions y el ×1.2 por specialty mapeada	components/AdvisorPanel.tsx:78,111-138 vs utils/actionEffects.ts:129-142, utils/actionCalculator.ts:24-26
C 26	"Estado: Inactivo por X turnos más"	Ningún asesor puede estar inactivo: siempre isActive:true	components/AdvisorPanel.tsx:121-123 vs engine/gameEngine.ts:375-379
C 27	Botones Reunirse/Negociar/Conceder sin precio	Costos reales ocultos al jugador: reunión \$10, negociar influencia×25, conceder influencia×45 (+ pasivas de gratis)	components/RightSidebar.tsx:463-486 vs utils/interactionCosts.ts:57-77
C 28	"Bloqueado {n}t" en interacciones	El lock es solo de UI: applyInteraction no verifica interactionHistory	components/RightSidebar.tsx:411-412,473-479 vs engine/gameEngine.ts:248-351
C 29	emitir_dinero: UI muestra "+3% popular." y riesgo derivado	Real: +1.2 de popularidad (×0.40); y con multiEffects.votingIntentionChange: −2 que se pierde al recalcularse la intención de voto (recalcState)	data/actionRegistry.ts:61-85 vs utils/actionEffects.ts:43,72; engine/turnProcessor.ts:337 + engine/engineShared.ts:120
C 30	Consejo de derrota hiperinflación: "no emitas más de 6 veces: la séptima desata la hiperinflación"	Correcto (7) — incluido como control: este texto sí coincide con el código (moneyPrintingCount >= 7), a diferencia de los demás umbrales	data/defeatReasons.ts:43-49 vs utils/victoryConditions.ts:76-78
C 31	Advertencia "Has emitido dinero N veces. A las 7 emisiones... perderás" dispara desde count >= 5 (critical) y "riesgo inflacionario" desde 3	Coincide con processInflation (≥3 inflación moderada, ≥5 crisis) — control OK	engine/turnProcessor.ts:64-81,231-258
C 32	LegacyScreen/"Resumen de Desempeño" y textos de careerLog narran "comenzó su carrera como Presidente" y ramas "promotion"	Coherente pero evidencia el modo campaña muerto: la narrativa de saltos de cargo (careerLog.ts:33-37) y "Cargos ocupados" siempre mono-cargo en MVP	utils/careerLog.ts:14-38,99-105
C 33	Comentario en SpecialAbilitiesPanel: "Hoy hay 1 por arquetipo"	Hay 2 habilidades por arquetipo (ARCHETYPE_ABILITIES) — comentario stale, el carrusel sí se usa	components/SpecialAbilitiesPanel.tsx:39-41 vs data/specialAbilities.ts:23-157
C 34	GameHeader re-exporta Users "por compat"	Exportación zombie sin consumidores	components/GameHeader.tsx:216-217

#	Qué dice la UI	Qué hace el código	Dónde (UI vs engine)
C 35	Tendencia de IndicatorsPanel: chips "—" fijos para estabilidad/legitimidad/conflicto/voto	No hay series históricas para esas variables (trends hardcodeado a null) — chip decorativo permanente	components/IndicatorsPanel.tsx:297-303
C 36	Pasiva sindicalista "Piso de contención: +1 acción base por apoyo popular"	Es +1 acción incondicional (extraActions: 1), no depende del apoyo popular	data/specialAbilities.ts:188 vs engine/archetypeEngine.ts:30-34, utils/actionCalculator.ts:21

Notas transversales para el documento de decisiones

- 1. **Derrotas realmente alcanzables en el MVP:** `low\_popularity` (<30 ×2), `negative\_budget` (<0 ×2), `hyperinflation` (7 emisiones), `election\_loss` (<45% en generales). `impeachment` está sombreado y `institutional\_coup` es imposible — dos de las seis rutas de `DefeatReason` son código muerto con textos vivos.
- 2. **Doble piso de popularidad:** `calculatePopularidad` (`utils/popularidad.ts:11-12`) usa `max(popularidad, 0.4·grupos + 0.6·pop)` ⇒ el apoyo grupal **solo puede subir**, nunca bajar, la popularidad; y `popularidadGrupos`/`popularidadPolitica` se calculan pero nadie los muestra.
- 3. **Fiscalía de datos:** `data/actionCategories.ts` es mencionado como "archivo zombie" en un comentario de `actionRegistry.ts:70-73` — el registry es la única fuente de verdad de acciones (verificaría borrado; no se usa en imports de engine, solo puede quedar referencia visual).
- 4. **Todo el balance de dificultad no-normal** (decay ×1.3/×1.5, ingreso ×0.85/×0.7, crisis ×1.5/×2.0, préstamos deshabilitados) es inerte al no existir selector; activarlo requiere UI, no engine.
- 5. Los "targets" de UI (`POSITION\_THRESHOLDS` 40/40 para presidente, target de conflicto 40, target de legitimidad 50) son **marcadores gráficos sin consecuencia** y en el caso de popularidad contradicen el umbral de derrota real (30).
- 6. Verificación de la premisa "legislativeSupport siempre null": **falsa** — se asigna en legislativas (año 2T4) y permanece fijo hasta la próxima elección; lo correcto es "null durante el año 1 de cada mandato y reseteado entre mandatos".